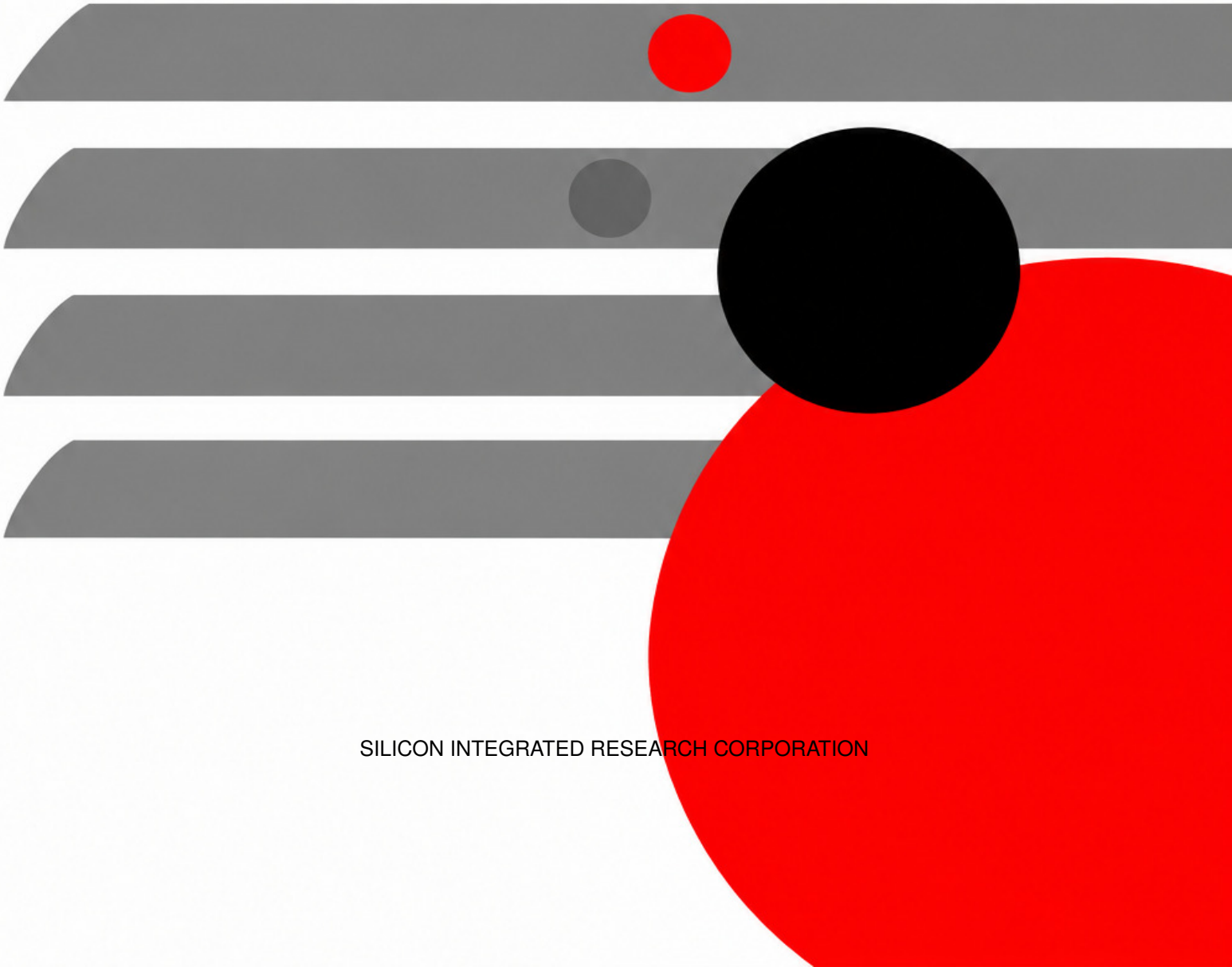


SIRCADA

Reference Manual

SIRC Analogue Display Adapter: Picture Processing Unit, Memory Map and Video Modes



SILICON INTEGRATED RESEARCH CORPORATION

Preface

About This Manual

This manual documents SIRCADA, the SIRC Analogue Display Adapter: the picture processing unit (PPU) that turns tile, tilemap and palette data in its own video memory into a 256 by 224 pixel picture. SIRCADA is the display adapter of the SIRCUS system, but it is a component in its own right, in a package of its own. It is the companion to the *SIRC-1 Architecture Reference Manual*, which defines the SIRC-1 CPU, to the *SIRC-1 Hardware Reference Manual*, which defines the CPU package and bus, and to the *SIRC-1 Assembly Programmer's Manual*, which defines the language used to program them.

SIRCADA is a peripheral on a 16-bit bus, and any 16-bit processor that can supply a chip select, a bus strobe and a read/write line can drive it. It was designed to work with the SIRC-1, so the signal names, the interrupt arrangements and the programming examples in this manual are those of the SIRC-1.

It is written for:

- Programmers writing graphics code for SIRCADA
- Authors of the tools that prepare tiles, tilemaps and palettes
- Engineers implementing the device, in hardware or in software

Assumed Knowledge

This manual assumes familiarity with tile-based two-dimensional graphics hardware: tilemaps, palettes, scanline rendering and the blanking intervals of a television signal. Readers who need the interrupt mechanism that delivers the vertical sync signal to the CPU should read the exception chapters of the *SIRC-1 Architecture Reference Manual*.

Revision Status

This is a draft of the *SIRCADA Reference Manual*, not for distribution. Anything not yet decided in the manual is marked “not yet specified”; the manual does not invent values for it.

Contents

Preface iii

1 Introduction 1
- 1.1 Purpose and Scope . . . 1
- 1.2 Device Overview . . . 1
- 1.3 Video Modes . . . 2
- 1.4 Block Diagrams . . . 2
- 1.5 Conventions . . . 3

2 Package and Pinout 5
- 2.1 Chip Layout . . . 5
- 2.2 Signal Descriptions . . . 6
  - 2.2.1 Clock Input (CLKI) . . . 7
  - 2.2.2 Reset Input ($\overline{\text{RST}}$) . . . 7
  - 2.2.3 Bus Signals . . . 7
  - 2.2.4 Interrupt Outputs ($\overline{\text{NMI}}$, $\overline{\text{IRQ4}}$) . . . 8
  - 2.2.5 Video Output . . . 8
  - 2.2.6 Power Pins (VCC1, VCC2, GND1, GND2) . . . 8
- 2.3 Connecting the Device . . . 8

3 Memory Map and Bus Access 9
- 3.1 The Device Segment . . . 9
- 3.2 Registers, Palette and Video Memory . . . 9
- 3.3 Access Rules . . . 9
  - 3.3.1 Access During Rendering . . . 10
  - 3.3.2 The Palette Boundary . . . 10
- 3.4 Planning a VRAM Layout . . . 10

4 Registers 11
- 4.1 Register Map . . . 11
- 4.2 Base Configuration Register . . . 13
- 4.3 Tile Size Register . . . 13
- 4.4 Tilemap and Tile Data Address Registers . . . 14
- 4.5 Scroll Registers . . . 14
- 4.6 Palette Configuration Registers . . . 14
- 4.7 Status Register . . . 15
- 4.8 Vertical Line Register . . . 15
- 4.9 Line Compare Register . . . 15
- 4.10 Interrupt Control Register . . . 15
- 4.11 Reset State . . . 16

5 Data Formats 17
- 5.1 Colour . . . 17
- 5.2 The Palette . . . 17
- 5.3 Tile Data . . . 17
  - 5.3.1 Mode 1 Tile Layout . . . 17
  - 5.3.2 Mode 2 Tile Layout . . . 17

5.3.3 16 by 16 Orthogonal Tiles . . . 17
5.3.4 Flipping Orthogonal Tiles . . . 18
5.3.5 2-Bit Orthogonal Tiles . . . 18
5.4 Tilemap Entries . . . 18
5.5 Tilemap Sizes . . . 19
5.6 Palette Selection . . . 19
5.7 Compositing Rule . . . 19

6 Mode 1: Orthogonal Backgrounds 21
6.1 Overview . . . 21
6.2 Backgrounds . . . 21
6.2.1 Scrolling . . . 21
6.3 Programming Mode 1 . . . 21
6.4 Priority and Compositing . . . 21
6.5 Fetch Schedule . . . 22

7 Mode 2: Isometric Backgrounds 23
7.1 Overview . . . 23
7.1.1 What the Mode Is For . . . 23
7.2 Geometry . . . 23
7.2.1 The Diamond Footprint . . . 24
7.3 Tilemap, Scrolling and Wrapping . . . 24
7.3.1 Moving Around the Map . . . 25
7.4 The Staircase Tile Format . . . 25
7.4.1 Locating a Texel . . . 25
7.4.2 Why It Works . . . 26
7.5 Fetch Schedule and Pipeline . . . 26
7.5.1 Slots . . . 26
7.5.2 Tilemap Entry Fetches . . . 26
7.5.3 Spare Slots . . . 26
7.5.4 Groups and Fine Scrolling . . . 27
7.5.5 Per-Line Sequence . . . 27
7.6 The Status Layer . . . 27
7.6.1 The Queue . . . 27
7.7 Two-Bit Tiles . . . 28
7.8 Shading . . . 28
7.9 Layers, Priority and Height . . . 29
7.10 Authoring Tiles . . . 30
7.11 Restrictions . . . 30
7.12 A Worked Example . . . 31
7.13 Programming Mode 2 . . . 31

8 Mode 3: Orthogonal Backgrounds over an Isometric One 33
8.1 Overview . . . 33
8.2 The Orthogonal Backgrounds . . . 33
8.2.1 Scrolling . . . 33
8.3 Fetch Schedule . . . 33
8.4 Differences from Mode 2 . . . 34
8.5 Restrictions . . . 34
8.6 Programming Mode 3 . . . 34

9 Sprites **35**
9.1 Overview . . . 35
9.2 Sprite Table Entry . . . 35
9.3 Sprite Tile Data . . . 35
9.4 Shadow Pixels . . . 36
9.5 Drawing a Line . . . 36
9.6 How Many Sprites . . . 36
9.6.1 Fetch Budget . . . 36
9.6.2 Using the Visible Slots as Well . . . 37
9.7 Sprites in Modes 2 and 3 . . . 37
9.8 Programming Sprites . . . 38
9.9 Notes . . . 38

10 Timing **39**
10.1 Clocks . . . 39
10.2 The CPU Clock . . . 39
10.3 Line Timing . . . 39
10.3.1 The First Line . . . 40
10.4 Frame Timing . . . 40
10.4.1 PAL . . . 41
10.5 Interrupts . . . 41
10.5.1 Vertical Sync . . . 41
10.5.2 Line Compare . . . 41
10.6 Fetch Budget . . . 42
10.7 Pixel Pipeline . . . 42

11 Signal Timing **45**
11.1 Reading the Figures . . . 45
11.2 Bus Cycles . . . 45
11.2.1 Writes . . . 45
11.2.2 Reads . . . 45
11.2.3 Consecutive Cycles . . . 46
11.2.4 The Start of Vertical Blanking . . . 46
11.3 Interrupt Timing . . . 47
11.3.1 Vertical Sync . . . 47
11.3.2 Line Compare . . . 47
11.4 Reset . . . 48
11.5 Video Output . . . 48
11.5.1 Horizontal Sync . . . 48
11.5.2 Vertical Sync . . . 49

12 Programming the Device **51**
12.1 Starting Up . . . 51
12.2 One Frame . . . 51
12.3 Programming Cautions . . . 52

A Quick Reference **53**
A.1 Registers . . . 53
A.2 Memory Map . . . 53
A.3 Formats . . . 53
A.4 Orthogonal Tiles . . . 54
A.5 Mode 1 Addresses . . . 54
A.6 Mode 2 Formulas . . . 54

List of Tables

1.1 The SIRC-1 and SIRCUS Manual Set . . . 1
1.2 Video Device Summary . . . 2
1.3 Video Modes . . . 2

2.1 SIRCADA 52-pin PLCC Pinout . . . 6
2.2 Signal Summary . . . 7

3.1 Video Device Memory Map . . . 9
3.2 An Example Mode 1 VRAM Layout . . . 10

4.1 Register Map . . . 12
4.2 Base Configuration Register Fields . . . 13
4.3 Tile Size Register Fields . . . 14
4.4 Palette Configuration Register Fields . . . 15
4.5 Interrupt Control Register Fields . . . 16
4.6 Registers with a Non-Zero Reset Value . . . 16

5.1 Tilemap Entry Fields . . . 18

6.1 Mode 1 Fetch Slots per Group of Four Pixels . . . 22

7.1 Rows of a Tile's Diamond Footprint (256 Pixels in All) . . . 24
7.2 Mode 2 Fetch Slots per Group of Four Pixels . . . 26
7.3 Shading Bits of an Isometric Tilemap Entry . . . 28
7.4 Register Values of the Example Scene . . . 31

8.1 Mode 3 Fetch Slots per Group of Four Pixels . . . 34

9.1 Sprite Table Entry . . . 35
9.2 Sprite Fetch Cost per Line, Blanking Only . . . 37

10.1 Clock Rates . . . 39
10.2 Line Timing . . . 40
10.3 Video Timing . . . 41
10.4 Fetch Slots per Group of Four Pixels . . . 42

A.1 Register Summary . . . 53

List of Figures

1.1 The Video Device in the System . . . 3
1.2 Blocks of the Picture Processing Unit . . . 3

2.1 SIRCADA 52-pin PLCC Pinout, Top View . . . 5
2.2 The Video Device in a System . . . 8

4.1 Base Configuration Register (`CONFIG`, Offset `0x0000`) . . . 13
4.2 Tile Size Register (`TILESIZE`, Offset `0x0001`) . . . 13
4.3 Scroll Registers (`B1SCRX`–`B4SCRY`) . . . 14
4.4 Palette Configuration Registers (`B1PAL`–`B4PAL`) . . . 14
4.5 Status Register (`STATUS`, Offset `0x0019`) . . . 15
4.6 Interrupt Control Register (`INTCTL`, Offset `0x0016`) . . . 15

5.1 Colour Format . . . 17
5.2 Four Pixels of Tile Data . . . 17
5.3 Tilemap Entry . . . 18

7.1 Fog of War and a Vehicle Shadow . . . 29
7.2 Mode 2 Output for the Example Scene . . . 31

9.1 Two Lorries and a Tree over the Mode 2 Demonstration Scene . . . 38

10.1 Timeline of One Visible Line, in Master Clocks . . . 40
10.2 Who Uses Each Fetch Slot of a Group of Four Pixels . . . 42
10.3 Fetch Schedule of One Group of Four Pixels (e: Tilemap Entry, w: Tile Word, ′: Second Cell) . . . 43

11.1 Write Cycle . . . 45
11.2 Read Cycle . . . 46
11.3 Three-Word DMA Write into the Device . . . 46
11.4 Video Memory Writes across the Start of Vertical Blanking (`VIE` set) . . . 47
11.5 Line-Compare Interrupt Raised at the Start of Line N . . . 47
11.6 Acknowledging the Line-Compare Interrupt with a Write to `LINE` . . . 48
11.7 Reset, with $\overline{\text{RST}}$ Held for the Minimum of Six Cycles . . . 48
11.8 Horizontal Sync and the Visible Part of a Line . . . 49
11.9 Vertical Sync in Lines 245 to 247 (NTSC), with Horizontal Sync before and after . . . 49

12.1 A Frame . . . 51

CHAPTER 1

Introduction

1.1 Purpose and Scope

The SIRC-1 CPU is sold on its own, so it is documented in three manuals that stand apart from any system built around it. SIRCADA is made for the SIRCUS system and has a manual of its own. Together the four manuals have the following scopes:

Manual	Scope
SIRC-1 Architecture Reference Manual	ISA, instruction encodings, canonical assembly notation, exceptions, programmer-visible state
SIRC-1 Assembly Programmer's Manual	Complete assembly syntax, directives, symbols, meta-instructions, and toolchain usage
SIRC-1 Hardware Reference Manual	Pinout, bus interface, timing, electrical characteristics
SIRCADA Reference Manual	The picture processing unit: package and pinout, memory map, registers, data formats, video modes, timing (this manual)

Table 1.1: The SIRC-1 and SIRCUS Manual Set

This manual covers SIRCADA as a component: its package and pins, what the CPU can read and write, how the device turns the contents of its video memory into a picture, and the timing of its signals. It does not describe the CPU, its instructions or its bus protocol. The device is a bus peripheral like any other, so the *SIRC-1 Hardware Reference Manual* is authoritative for how the CPU makes a bus cycle, and this manual is authoritative for how the device answers it and for what it does once it has been reached.

1.2 Device Overview

The video device draws a picture one scanline at a time, with no frame buffer: for every group of four pixels it fetches tilemap entries and tile data from its video memory, resolves them through a palette, and sends the result to the display.

Property	Value
Picture size	256 by 224 pixels, the first line always blank (262 lines NTSC, 312 PAL; Chapter 10)
Colour	15 bits per pixel, 5 bits for each of red, green and blue
Palette	256 entries of 16 bits, directly addressable by the CPU
Video memory	32,768 words (64 KB), shared by tile data and tilemaps
Tile data	4 bits per pixel
Backgrounds	Four orthogonal in Mode 1; two isometric and one orthogonal in Mode 2; two orthogonal and one isometric in Mode 3
Sprites	128 entries, three layers, 32 per line (Chapter 9)
Interrupts	Two: vertical sync (NMI) and line compare (IRQ4)
Line timing	1364 master clocks per line, 262 lines per frame

Table 1.2: Video Device Summary

1.3 Video Modes

The device has three video modes, selected by the `MODE` field of the Base Configuration register (Section 4.2). The modes share the memory map, the registers, the palette and the data formats. They differ in how the backgrounds are drawn and in the fetch schedule that supports them.

Mode	`MODE` value	Description
Mode 1	0	Four orthogonal backgrounds of square tiles and sprites. The reset mode.
Mode 2	1	Two isometric backgrounds: square tiles displayed as diamonds on a 2:1 ground plane, and an orthogonal status layer.
Mode 3	2	One isometric background behind two orthogonal backgrounds.

Table 1.3: Video Modes

Mode 1 is the conventional mode and is described in Chapter 6. Mode 2 is a specialised mode for games set on a flat map seen from above at an angle. It is good at that and at little else; Chapter 7 describes what it does and, as importantly, what it gives up. Mode 3 (Chapter 8) keeps one isometric background and gives the fetch slots of the other to two orthogonal ones. The values 3–7 of `MODE` are reserved; software must not write them. The `MODE` field is sampled at the first cycle of each line, so a change takes effect at the start of the next line.

1.4 Block Diagrams

Figure 1.1 shows the device as the CPU sees it: four areas of memory behind one bus interface, a video output, and two interrupt lines. Figure 1.2 shows what is inside. The chapters that follow describe each block; the figure says how they feed each other.

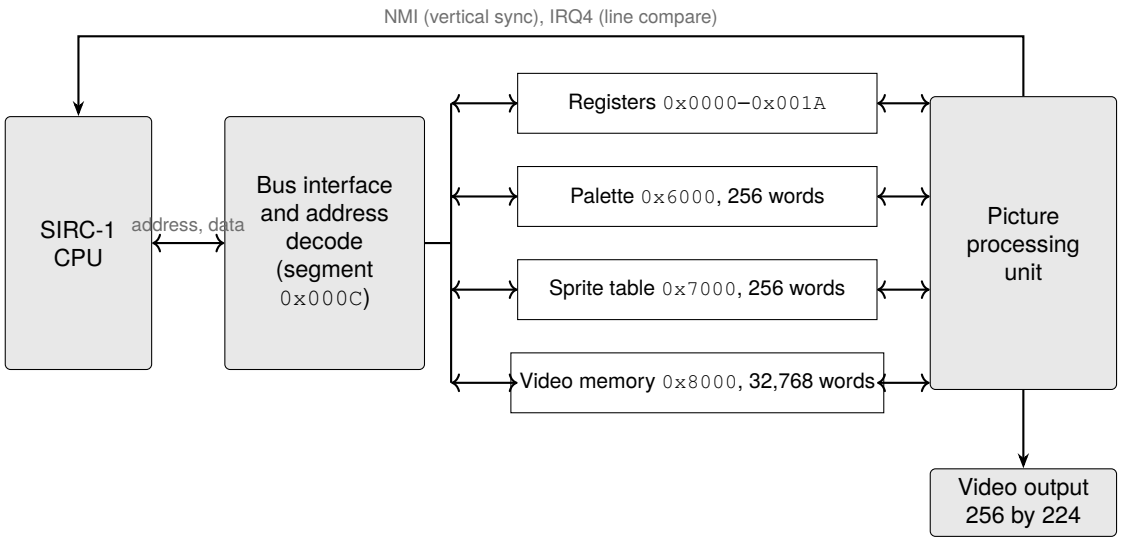


Figure 1.1: The Video Device in the System

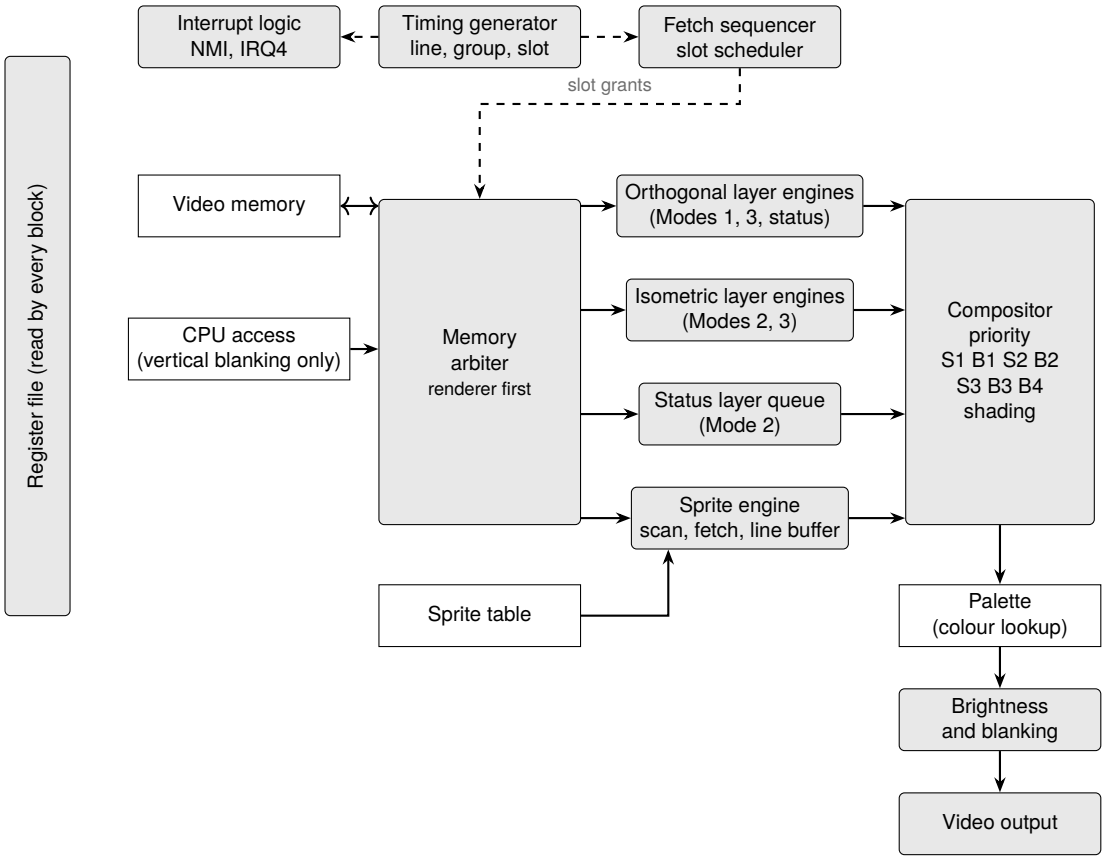


Figure 1.2: Blocks of the Picture Processing Unit

1.5 Conventions

- All addresses and sizes are in 16-bit words, as on the CPU. A word address is an offset from the base of the device's segment unless stated otherwise.
- Bit 0 is the least significant bit. Bit diagrams draw the most significant bit on the left.
- Hexadecimal values carry a `0x` prefix and upper case digits.

- Ranges of values and addresses use an en dash: `0x0000–0x00FF`.
- **Reserved** fields and registers are set aside by the architecture. Software must not rely on what they read, and should write zero to a reserved field it does not otherwise use.
- A value that the design has not fixed is **not yet specified**. It is not undefined, and no value is implied.

CHAPTER 2

Package and Pinout

The video device is a single integrated circuit. The picture processing unit, the video memory, the palette, the sprite table and the colour converters are all on the one piece of silicon, so the only signals that leave the package are the bus, two interrupt outputs, the clock and reset inputs, and the video output. This chapter gives the package and the meaning of every pin. Chapter 11 gives the timing of the signals.

2.1 Chip Layout

The device is supplied in a 52-pin plastic leaded chip carrier (PLCC-52), 13 pins on each side. Pin 1 is at the centre of the top edge, marked by a dot on the package; the pins are numbered counter-clockwise when the package is seen from above.

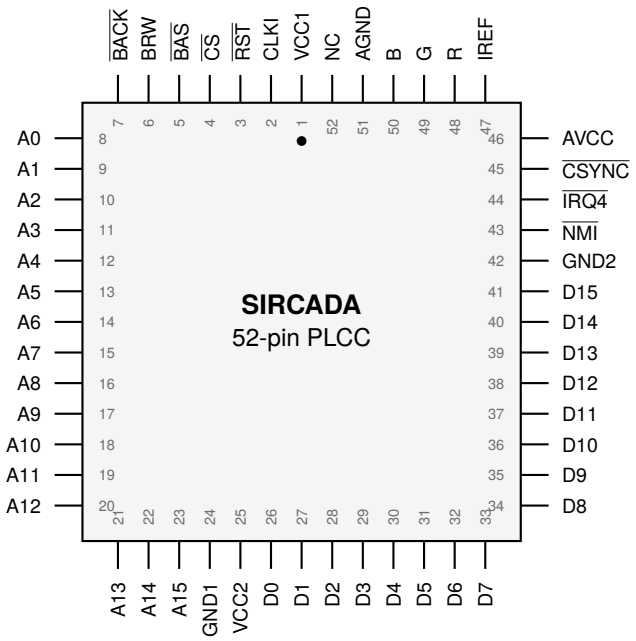


Figure 2.1: SIRCADA 52-pin PLCC Pinout, Top View

Pin	Name	Description
1	VCC1	Power supply
2	CLKI	Clock input
3	$\overline{\text{RST}}$	Reset input
4	$\overline{\text{CS}}$	Chip select input
5	$\overline{\text{BAS}}$	Bus access strobe input
6	BRW	Bus read/write input
7	$\overline{\text{BACK}}$	Bus acknowledge output
8–23	A0–A15	Address bus bits 0–15
24	GND1	Ground
25	VCC2	Power supply
26–41	D0–D15	Data bus bits 0–15
42	GND2	Ground
43	$\overline{\text{NMI}}$	Vertical sync interrupt output
44	$\overline{\text{IRQ4}}$	Line-compare interrupt output
45	$\overline{\text{CSYNC}}$	Composite sync output
46	AVCC	Analog power supply
47	IREF	Colour output reference
48–50	R, G, B	Analog colour outputs
51	AGND	Analog ground
52	NC	No connection

Table 2.1: SIRCADA 52-pin PLCC Pinout

2.2 Signal Descriptions

Table 2.2 lists the direction and sense of each signal. Active-low signals carry a bar over the name; they are asserted when low. Address and data signals are active high. An output marked *open drain* can only pull its line low; the system supplies a pull-up, and several devices may share the line.

Signal	Pins	Direction	Output type	Function
CLKI	2	Input		Rising edge advances the device one PPU cycle
$\overline{\text{RST}}$	3	Input		Resets the device
$\overline{\text{CS}}$	4	Input		Selects the device for a bus cycle
$\overline{\text{BAS}}$	5	Input		Marks a bus cycle
BRW	6	Input		High for a read, low for a write
$\overline{\text{BACK}}$	7	Output	Open drain	Completes the bus cycle
A0–A15	8–23	Input		Word offset in the device segment
D0–D15	26–41	Bidirectional	Three-state	Data
$\overline{\text{NMI}}$	43	Output	Open drain	Vertical sync interrupt
$\overline{\text{IRQ4}}$	44	Output	Open drain	Line-compare interrupt
$\overline{\text{CSYNC}}$	45	Output	Driven	Composite sync
R, G, B	48–50	Output	Analog	Colour
IREF	47	Reference		Sets the full-scale colour level

Table 2.2: Signal Summary

2.2.1 Clock Input (CLKI)

The device has one clock input, the same `CLKI` the system supplies to the CPU. One `CLKI` cycle is one PPU cycle: the master clock divided by two (Section 10.2). Every internal interval, the pixel, the group of four pixels, the line and the frame, is a whole number of `CLKI` cycles, so the device needs no other clock. The pixel rate is `CLKI` divided by two. The rising edge is the only edge the device uses. The range of clock frequencies is not yet specified.

2.2.2 Reset Input ($\overline{\text{RST}}$)

While $\overline{\text{RST}}$ is asserted the device is held in its reset state (Section 11.4). The input is sampled on each rising edge of `CLKI`, and must stay asserted for at least six cycles. The $\overline{\text{RSTO}}$ output of the CPU holds for at least that long and can be connected directly.

2.2.3 Bus Signals

The device is a bus peripheral. It watches the bus of the CPU (*SIRC-1 Hardware Reference Manual*) and answers the cycles that are addressed to it. It has no byte strobes and no access-type inputs: it treats an instruction fetch, a data access and a DMA transfer alike.

$\overline{\text{CS}}$ The system decodes the segment of the device, the upper address bits that the device does not see, and asserts $\overline{\text{CS}}$ while the segment is addressed. A cycle in which $\overline{\text{CS}}$ or $\overline{\text{BAS}}$ is deasserted is ignored: the device drives neither the data bus nor $\overline{\text{BACK}}$.

$\overline{\text{BAS}}$, BRW The CPU asserts $\overline{\text{BAS}}$ for a whole cycle that makes a bus access. BRW is high for a read and low for a write. Both are valid for the whole of the cycle.

A0–A15 The word offset of the access within the segment (Chapter 3). The device accesses one 16-bit word per cycle.

D0–D15 On a write the CPU drives the bus and the device takes the word at the rising edge that ends the cycle. On a read the device drives the word that the offset selects, from the start of the cycle to its end, and releases the bus when the cycle ends. A read of an offset that has no register or memory is driven as zero.

$\overline{\text{BACK}}$ The device asserts $\overline{\text{BACK}}$ for the whole of every cycle in which $\overline{\text{CS}}$ and $\overline{\text{BAS}}$ are asserted, whatever the offset and whatever the state of the picture. It never asks the CPU to wait and never raises a bus error. $\overline{\text{BACK}}$ is open drain, so it may be joined with the acknowledge from other devices on the bus.

2.2.4 Interrupt Outputs (NMI, IRQ4)

The vertical sync interrupt is a one-cycle pulse on NMI, and the line-compare interrupt is a level held on IRQ4 (Section 10.5). Both are open drain, so the line is shared with other devices when the system connects them; a program that shares IRQ4 reads the `LIP` bit of the Status register to see whose interrupt it is. Neither output is driven while RST is asserted.

2.2.5 Video Output

CSYNC Composite sync. It is asserted for the horizontal sync pulse of every line and, in the lines of vertical sync, for the vertical sync pulses (Section 11.5).

R, G, B The picture, as three analog levels. Each is converted from 5 bits of the colour of the pixel after the brightness setting has been applied, so it has 32 steps from black to full scale. In blanking, in forced blank and during reset the three outputs are at black.

IREF A resistor from IREF to AGND sets the level of full scale for all three outputs. The value of the resistor and the voltages of the outputs are not yet specified.

AVCC, AGND The supply of the converters, kept apart from VCC1, VCC2, GND1 and GND2 so that the noise of the logic does not reach the picture. AVCC is 5 V; AGND is connected to the ground of the logic at one point on the board.

2.2.6 Power Pins (VCC1, VCC2, GND1, GND2)

The four pins supply the logic and are all connected. VCC is 5 V ± 5%, as for the CPU. The electrical characteristics other than the supply are not yet specified.

2.3 Connecting the Device

Figure 2.2 shows the device in a system. The CPU and the video device share the address bus (the device uses the lower 16 bits), the data bus, BAS, BRW, BACK and `CLKI`. The address decoder asserts CS for the segment of the device. The reset output of the CPU resets the device. The interrupt outputs of the device go to the interrupt inputs of the CPU, with pull-up resistors, and the video outputs go to the display connector.

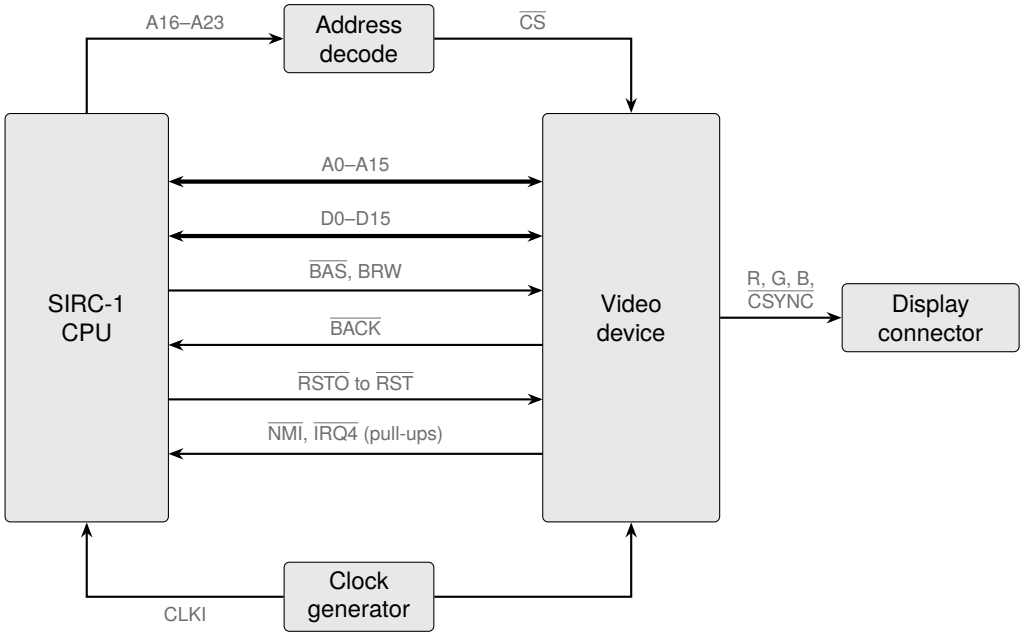


Figure 2.2: The Video Device in a System

CHAPTER 3

Memory Map and Bus Access

3.1 The Device Segment

The video device occupies one segment of the CPU's address space, 65,536 words long. The segment is chosen by the system; the SIRCUS system maps the device at segment `0x000C`, so a register at offset `0x0002` is at the CPU address `0x000C_0002`. This manual gives every address as an offset from the base of the segment.

Offsets	Region	Notes
`0x0000–0x00FF`	Registers	Offsets `0x0000–0x001A` are defined (Chapter 4); the rest are open bus
`0x0100–0x5FFF`	Unmapped	Open bus
`0x6000–0x60FF`	Palette	256 entries, one word each (Section 5.2)
`0x6100–0x6FFF`	Unmapped	Open bus; see Section 3.3.2
`0x7000–0x70FF`	Sprite table	256 words, 128 entries of two words (Chapter 9)
`0x7100–0x7FFF`	Unmapped	Open bus
`0x8000–0xFFFF`	Video memory	32,768 words of tile data and tilemaps

Table 3.1: Video Device Memory Map

3.2 Registers, Palette and Video Memory

The **registers** control the device. They are described in Chapter 4.

The **palette** holds 256 colours in a small, fast memory that the renderer reads once per pixel. It can be read and written by the CPU at any time.

The **video memory** (VRAM) holds everything else: tile data, tilemaps and, when they are specified, sprite tile data. It is 32,768 words, which is 64 KB. The device does not interpret VRAM in any fixed layout. Each background has a base address register for its tilemap and another for its tile data, and any VRAM range may hold either. The only constraint is that ranges the renderer reads must not overlap ranges the program expects to stay untouched.

An address in VRAM is written as its offset in the segment, so VRAM begins at `0x8000`. The base address registers hold addresses in this form: a tilemap at the start of VRAM has the base address `0x8000`, not `0x0000`.

3.3 Access Rules

A read of a register, a palette entry or a VRAM word returns the stored value. A write stores the value. A read of an open bus offset returns zero and a write to one is ignored. The device never inserts a wait state: every access completes in the cycle it is made, whether or not it takes effect (Section 3.3.1).

The Status register (offset `0x0019`) and the Vertical Line register (offset `0x001A`) are read-only; a write to either is ignored.

3.3.1 Access During Rendering

The renderer reads video memory in the visible lines, and in the horizontal blanking interval too: the sprites of the next line are fetched there (Chapter 9). So video memory and the sprite table are available to the CPU only in vertical blanking, the lines after the last visible line (Chapter 10), or while the picture is turned off (`GD` in the Base Configuration register is 1), when the device makes no reads at all:

- A write to video memory or to the sprite table at any other time is ignored, and a read returns zero. At reset `GD` is 1, so a program can load video memory straight away and then clear `GD`.
- The palette and the registers can be read and written at any time. A palette write changes the colour of the pixels drawn after it, which allows a palette change in the middle of the picture. A register write takes effect when the device next samples that register, normally at the start of the next line (Chapters 7 and 8).

3.3.2 The Palette Boundary

The palette occupies 256 words, `0x6000`–`0x60FF`. The offset `0x6100` and the rest of the range up to `0x6FFF` are open bus.

3.4 Planning a VRAM Layout

Nothing fixes where tile data and tilemaps live, so a program chooses. One arrangement puts the tile set at the bottom of VRAM and the tilemaps at the top, leaving room for the tile set to grow:

Contents	Offsets	Size
Tile set (Mode 1)	`0x8000` and up	16 words per tile
Background 1 tilemap	`0xF000`	1024 words
Background 2 tilemap	`0xF400`	1024 words
Background 3 tilemap	`0xF800`	1024 words
Background 4 tilemap	`0xFC00`	1024 words

Table 3.2: An Example Mode 1 VRAM Layout

A tilemap is a 32 by 32 table of one-word entries, so it takes 1024 words. A Mode 1 tile of 8 by 8 pixels at 4 bits per pixel takes 16 words; a Mode 2 tile of 16 by 16 texels takes 64 words (Section 7.4). VRAM can therefore hold at most 2048 Mode 1 tiles or 512 Mode 2 tiles, before any tilemap is placed in it.

CHAPTER 4

Registers

4.1 Register Map

The registers occupy the first words of the device segment. Every defined register is 16 bits wide and is read and written a word at a time.

Offset	Name	Access	Description
0x0000	CONFIG	R/W	Base configuration: mode, layer disables, brightness
0x0001	TILESIZE	R/W	Tile and tilemap sizes
0x0002	B1MAP	R/W	Background 1 tilemap address
0x0003	B2MAP	R/W	Background 2 tilemap address
0x0004	B3MAP	R/W	Background 3 tilemap address
0x0005	B4MAP	R/W	Background 4 tilemap address
0x0006	B1TILES	R/W	Background 1 tile data address
0x0007	B2TILES	R/W	Background 2 tile data address
0x0008	B3TILES	R/W	Background 3 tile data address
0x0009	B4TILES	R/W	Background 4 tile data address
0x000A	B1SCRX	R/W	Background 1 horizontal scroll
0x000B	B2SCRX	R/W	Background 2 horizontal scroll
0x000C	B3SCRX	R/W	Background 3 horizontal scroll
0x000D	B4SCRX	R/W	Background 4 horizontal scroll
0x000E	B1SCRY	R/W	Background 1 vertical scroll
0x000F	B2SCRY	R/W	Background 2 vertical scroll
0x0010	B3SCRY	R/W	Background 3 vertical scroll
0x0011	B4SCRY	R/W	Background 4 vertical scroll
0x0012	B1PAL	R/W	Background 1 palette configuration
0x0013	B2PAL	R/W	Background 2 palette configuration
0x0014	B3PAL	R/W	Background 3 palette configuration
0x0015	B4PAL	R/W	Background 4 palette configuration
0x0016	INTCTL	R/W	Interrupt enables
0x0017	LINE	R/W	Line-compare interrupt line; a write acknowledges
0x0018	STILES	R/W	Sprite tile data address
0x0019	STATUS	R	Version, output mode, sprite overflow, vertical blanking
0x001A	VLINE	R	Current line
0x001B–0x00FF	—	Open bus	Reads zero, writes ignored

Table 4.1: Register Map

The register names are those of this manual. The registers are laid out in groups of four, one for each background, so a register for background n is at the group's base plus $n - 1$.

4.2 Base Configuration Register

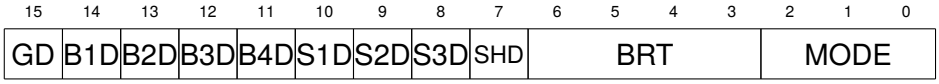


Figure 4.1: Base Configuration Register (`CONFIG`, Offset `0x0000`)

Bits	Field	Meaning
15	`GD`	Graphics disable (forced blank): when 1 the picture is black and the device makes no reads, so video memory and the sprite table can be accessed at any time. 1 at reset
14, 13, 12, 11	`B1D`–`B4D`	Disable background 1, 2, 3, 4 when 1 (Sections 7.9 and 6.4). Background 4 is drawn in Mode 1 only
10, 9, 8	`S1D`, `S2D`, `S3D`	Disable sprite layer 1, 2, 3 when 1 (Chapter 9)
7	`SHD`	Sprite shadow enable: when 1, value 15 of sprite palette 7 is a shadow (Section 9.4); 0 at reset
6–3	`BRT`	Screen brightness: each colour channel of the final picture is multiplied by $(\texttt{BRT}+1)/16$, so 15 is full brightness and 0 nearly black. `GD` blanks the picture completely
2–0	`MODE`	Video mode: 0 for Mode 1, 1 for Mode 2, 2 for Mode 3, others reserved

Table 4.2: Base Configuration Register Fields

The `MODE` field is sampled at the first cycle of each line, so a change takes effect at the start of the next line. Software should change mode during vertical blanking.

4.3 Tile Size Register

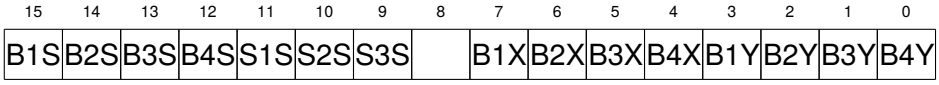


Figure 4.2: Tile Size Register (`TILESIZE`, Offset `0x0001`)

Bits	Field	Meaning
15–12	B1S–B4S	Orthogonal background tile size: 0 for 8 by 8, 1 for 16 by 16 (Section 5.3.3)
11, 10, 9	S1S, S2S, S3S	Sprite tile size, same encoding
7–4	B1X–B4X	Tilemap width: 0 for one screen of 32 entries, 1 for two (Section 5.5)
3–0	B1Y–B4Y	Tilemap height: 0 for one screen of 32 entries, 1 for two
8	—	Reserved

Table 4.3: Tile Size Register Fields

An *orthogonal* background is an ordinary background of square tiles in rows and columns, as opposed to an isometric one. The word is used for all of them: the four backgrounds of Mode 1, the two of Mode 3 and the status layer of Mode 2. Orthogonal backgrounds scroll, except for the status layer of Mode 2. The tile size bits B1S–B4S apply to orthogonal backgrounds, that is, to all four backgrounds in Mode 1 and to backgrounds 1 and 2 in Mode 3, but not to the status layer, which always uses 8 by 8 tiles. The tilemap size bits apply to orthogonal and isometric backgrounds. Isometric backgrounds always use 16 by 16 texel tiles and ignore the tile size bits (Section 7.11).

4.4 Tilemap and Tile Data Address Registers

B1MAP–B4MAP hold the word address of the first entry of the background's tilemap. B1TILES–B4TILES hold the word address of the first word of tile number 0 of its tile data. Both are full offsets within the segment, so a value for VRAM is at least 0x8000 (Chapter 3). All 16 bits are significant. The renderer adds the offset of an entry or tile to the base with 16-bit wraparound.

STILES holds the sprite tile data address (Chapter 9).

4.5 Scroll Registers

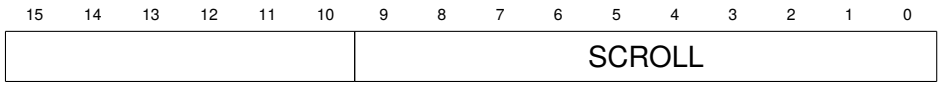


Figure 4.3: Scroll Registers (B1SCRX–B4SCRY)

Bits 9–0 hold the scroll amount, 0 to 1023, in pixels. Bits 15–10 are reserved. The amount moves the background's view of its map: increasing SCRX moves the picture to the left, and increasing SCRY moves it up.

Every background of every mode scrolls. The amounts are sampled once per line, as the chapter of each mode describes.

4.6 Palette Configuration Registers

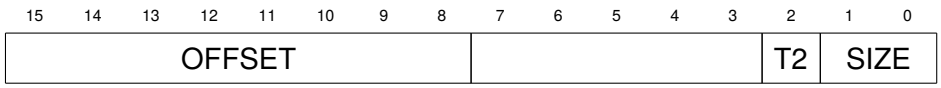


Figure 4.4: Palette Configuration Registers (B1PAL–B4PAL)

Bits	Field	Meaning
15–8	OFFSET	Palette entry added to every colour address of the background
7–3	—	Reserved
2	T2	2-bit tiles (Sections 7.7 and 5.3.5)
1–0	SIZE	Colours per palette: 0 for 4, 1 for 8, 2 for 16, 3 for 32

Table 4.4: Palette Configuration Register Fields

Section 5.6 explains how OFFSET and SIZE combine with a tilemap entry's palette selection to choose a palette entry. 4-bit tile data uses sizes of 16 colours or fewer.

4.7 Status Register

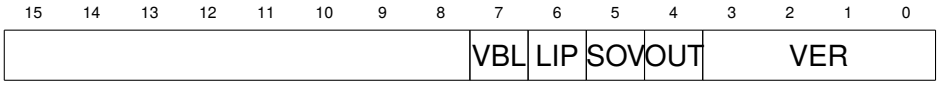


Figure 4.5: Status Register (STATUS, Offset 0x0019)

VER, bits 3–0, is the version of the device, which is 1. OUT, bit 4, is the output mode: 0 for NTSC and 1 for PAL. Both are fixed when the device is made. SOV, bit 5, is the sprite overflow flag: it is set when any line of the frame had more than 32 sprites (Section 9.6), and cleared at the start of line 0 of the next frame, so a program reads it in vertical blanking. LIP, bit 6, is set while a line-compare interrupt is pending (Section 10.5.2). VBL, bit 7, is set while the beam is in vertical blanking, from the first cycle of line 224 to the last cycle of the last line of the frame, and is clear in lines 0 to 223; it is the flag a program polls to know that video memory is open (Section 3.3.1). Bits 15–8 are reserved. The register is read-only.

4.8 Vertical Line Register

VLINE, offset 0x001A, holds in bits 8–0 the number of the line the beam is on: 0 to 261 on NTSC and 0 to 311 on PAL. It changes at the first cycle of each line, the cycle at which a line-compare interrupt for that line is raised, and bits 15–9 read as zero. The register is read-only. A program reads it to find where in the frame it is, and reads it twice, or compares it with LINE, when a line may have passed between two accesses.

4.9 Line Compare Register

LINE holds, in bits 8–0, the number of the line at which the line-compare interrupt fires when LIE is set in the Interrupt Control register (Section 10.5.2). Bits 15–9 are reserved. A write to LINE, of any value, also acknowledges a pending line-compare interrupt.

4.10 Interrupt Control Register

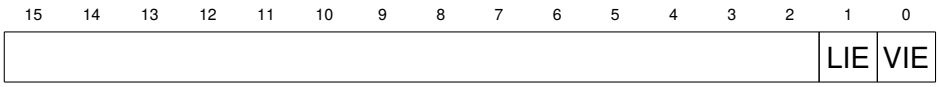


Figure 4.6: Interrupt Control Register (INTCTL, Offset 0x0016)

Bits	Field	Meaning
15–2	—	Reserved
1	LIE	Line interrupt enable: when 1, the line-compare interrupt can be raised (Section 10.5.2)
0	VIE	Vertical sync interrupt enable: when 1, the device raises the NMI at the start of vertical blanking

Table 4.5: Interrupt Control Register Fields

Both bits are 0 at reset, so no interrupt is raised until the program has set up its handlers.

4.11 Reset State

At reset the device is blank and every writable register is zero, except as follows:

Register	Value at reset	Meaning
CONFIG	0x8078	GD = 1 (picture blank), BRT = 15, MODE = 0
STATUS	0x0001 (NTSC), 0x0011 (PAL)	VER = 1 and OUT fixed by the device; read-only

Table 4.6: Registers with a Non-Zero Reset Value

The contents of video memory, the palette and the sprite table are undefined at reset. A program must initialise whatever it uses, and then clear GD, before the picture is shown. A program should hide every sprite it does not use (layer 3) before it clears GD.

CHAPTER 5

Data Formats

5.1 Colour

A colour is a 16-bit word with 5 bits for each of red, green and blue. Bit 15 is a reserved flag.

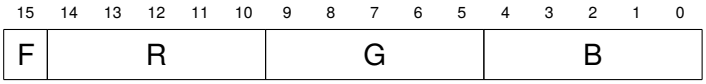


Figure 5.1: Colour Format

Each 5-bit channel drives the converter of its colour output (Chapter 2), which has 32 evenly spaced levels from black to full scale. Channel value 31 is full scale, so a pixel of 31 in every channel is full-scale white.

5.2 The Palette

The palette holds 256 colours at offsets `0x6000`–`0x60FF` (Chapter 3). Every pixel of every background is a palette lookup. Palette entry 0 is special in one way: it is the backdrop, the colour shown where no layer has a visible pixel.

5.3 Tile Data

A tile is a small square image of 4-bit pixels, each of which is a number from 0 to 15. The value 0 is transparent: the pixel shows whatever is behind it. The values 1–15 select a colour through the palette (Section 5.6).

5.3.1 Mode 1 Tile Layout

Four pixels fill one 16-bit word, with the leftmost pixel in the most significant nibble.

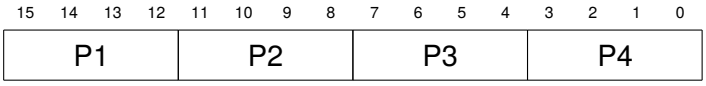


Figure 5.2: Four Pixels of Tile Data

An 8 by 8 tile is stored as eight rows of two words each, top row first, so it occupies 16 words. The word for pixels $4h$ to $4h + 3$ of row r of tile t is at the tile data base address plus $16t + 2r + h$, where h is 0 for the left half of the row and 1 for the right half. A 16 by 16 tile is described in Section 5.3.3.

5.3.2 Mode 2 Tile Layout

Mode 2 stores a 16 by 16 texel tile in a different arrangement, described in Section 7.4. It takes 64 words, the same as a 16 by 16 tile would in the row-by-row layout, but the words are not rows.

5.3.3 16 by 16 Orthogonal Tiles

An orthogonal background whose tile size bit (Section 4.3) is 1 uses 16 by 16 tiles. A 16 by 16 tile is stored as four 8 by 8 quadrants in the order top left, top right, bottom left, bottom right, each laid out as in Section 5.3.1, so a tile is 64 words. For the pixel in column c and row r of tile t the word is at

```
tile data base + 64 × tile number + 16 × quadrant + 2 × (row mod 8) +
half
```

where the quadrant is $2\lfloor r/8 \rfloor + \lfloor c/8 \rfloor$ and the half is $\lfloor (c \bmod 8)/4 \rfloor$. A 2-bit 16 by 16 tile is four quadrants of 8 words, 32 words, with the word at `base + 32 × tile + 8 × quadrant + (row mod 8)`. A tile of 64 words is the same size as an isometric tile, so one tile number addresses either. With 16 by 16 tiles a 32 by 32 tilemap covers 512 by 512 pixels. Each background chooses its own tile size, and backgrounds of different tile sizes need no further rule: each is fetched independently.

5.3.4 Flipping Orthogonal Tiles

When the `H` bit of an orthogonal tilemap entry is set, the tile is mirrored left to right: pixel column c is read from column $w - 1 - c$ of the tile, where w is 8 or 16. When `V` is set, row r is read from row $w - 1 - r$. The device reads the mirrored word and reverses the order of its pixels.

5.3.5 2-Bit Orthogonal Tiles

A background whose `T2` bit (Section 4.6) is set uses 2-bit tiles. An 8 by 8 orthogonal tile is then 8 words, one for each row, holding the row's eight pixels with the leftmost in bits 15–14:

```
tile data base + 8 × tile number + row
```

Value 0 is transparent, so a pixel has three colours. A word covers eight pixels, so the device reads it for every second group of four. 2-bit tiles apply to every orthogonal background and to the status layer. Isometric 2-bit tiles have a layout of their own (Section 7.7).

5.4 Tilemap Entries

A tilemap is a table of one-word entries, 32 entries to a row and 32 rows, stored row by row. An entry names a tile and how it is drawn.

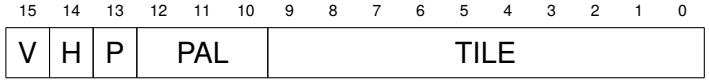


Figure 5.3: Tilemap Entry

Bits	Field	Meaning
15	`V`	Flip the tile vertically on an orthogonal layer (Section 5.3.3); darkens the tile in an isometric layer (Section 7.8)
14	`H`	Flip the tile horizontally on an orthogonal layer; lightens the tile in an isometric layer
13	`P`	Priority: 1 puts the tile in front of everything, sprite layer 1 included; 0 leaves its layer in the order S1, B1, S2, B2, S3, B3 (and B4 in Mode 1)
12–10	`PAL`	Palette selection, 0 to 7 (Section 5.6)
9–0	`TILE`	Tile number, 0 to 1023, counted from the background's tile data base address

Table 5.1: Tilemap Entry Fields

Tile numbers can reach 1023, but VRAM is not that large for every tile size: 64-word tiles stop at 512 in a VRAM with nothing else in it (Chapter 3). A tile number whose data lies past the end of the address space wraps at 16 bits.

5.5 Tilemap Sizes

A tilemap is made of screens of 32 by 32 entries, stored row by row, 1,024 words each. The width bit `BnX` and the height bit `BnY` of the Tile Size register (Section 4.3) double the map across or down:

`BnX`	`BnY`	Map
0	0	One screen, 32 by 32 entries
1	0	Two screens side by side, 64 by 32: A B
0	1	Two screens one above the other, 32 by 64: A over B
1	1	Four screens, 64 by 64: A B over C D

The screens are in memory in the order A, B, C, D, starting at the tilemap base. For the entry in column c and row r of a map W entries wide and H high (both after wrapping, $c \bmod W$ and $r \bmod H$), the address is

```
tilemap base + 1024 × screen + 32 × (r mod 32) + (c mod 32)
```

where the screen is $\lfloor r/32 \rfloor \cdot (W/32) + \lfloor c/32 \rfloor$. A map wraps at its size. The tilemap sizes apply to orthogonal backgrounds, where an entry is a tile, and to isometric backgrounds, where an entry is a cell (Section 7.3).

5.6 Palette Selection

A background pixel with a non-zero value v in a tile whose entry has palette selection s reads palette entry

$$\text{entry} = ((s \ll k) + \texttt{OFFSET} + v) \bmod 256,$$

where `OFFSET` and k come from the background's palette configuration register (Section 4.6): k is 2, 3, 4 or 5 for `SIZE` values 0, 1, 2 and 3, that is, for palettes of 4, 8, 16 or 32 colours. The sum wraps at 8 bits. A pixel whose value is 0 is transparent and reads no palette entry.

With 16-colour palettes, `OFFSET` 0 and a selection of 3, the value 5 reads palette entry 53. The eight selections therefore pick eight palettes of 16 colours, 128 palette entries in all, starting at the offset.

5.7 Compositing Rule

Layers are combined pixel by pixel. For each screen pixel the device takes the first layer, in front-to-back order, whose pixel value is not 0, and shows the palette entry that pixel selects. If every layer is transparent, the device shows palette entry 0. There is no blending: a pixel is either transparent or fully opaque.

The front-to-back order of the layers is S1, B1, S2, B2, S3, B3, B4 in Mode 1, the same without B4 in Mode 3, and S1, B1, S2, B2, S3 with a status layer in Mode 2; see Sections 6.4 and 7.9. A tile with its priority bit set is in front of everything.

CHAPTER 6

Mode 1: Orthogonal Backgrounds

6.1 Overview

Mode 1 is the device's conventional mode, the one it is in at reset. It draws up to four backgrounds of square tiles laid out in rows and columns, and three layers of sprites. The tiles of a background are 4 bits per pixel, or 2 bits when its `T2` bit is set (Section 5.3.5).

6.2 Backgrounds

Each background is a tilemap (Section 5.5), 32 by 32 entries by default, and a set of tiles of 8 by 8 or 16 by 16 pixels (Section 5.3). Each of the four chooses its own tile size and tilemap size, so a background covers from 256 by 256 pixels (8 by 8 tiles, one screen) to 1024 by 1024 (16 by 16 tiles, four screens), and the 256 by 224 picture is a window onto it. Each background also has its own scroll registers and palette configuration.

For the pixel at screen column x and line y, the renderer reads, for a background with scroll (s_x, s_y) and tiles w pixels square, the tilemap entry in column $\lfloor (x+s_x)/w \rfloor$ and row $\lfloor (y+s_y)/w \rfloor$, both taken modulo the size of the map, and then the word of tile data holding the pixel (Sections 5.3.1 and 5.3.3). It does this for all four backgrounds at once.

6.2.1 Scrolling

Scrolling works as it does for the orthogonal backgrounds of Mode 3 (Section 8.2.1): the scroll registers are sampled once per line, groups of four pixels are aligned to the plane, a window of two groups with an offset of $s_x \bmod 4$ gives fine scrolling, and a map wraps at its size.

6.3 Programming Mode 1

1. Write the palette, the tile data and the tilemaps into video memory, in vertical blanking or while `GD` is 1.
2. Write the `TILESIZE` register with the tile size and tilemap size of each background.
3. Write the tilemap address, tile data address and palette configuration registers of each background in use.
4. Write the Base Configuration register with `MODE` = 0, and with the disable bit of each unused background set.
5. Each frame, during vertical blanking, write the scroll registers and any tilemap entries that have changed.

6.4 Priority and Compositing

The front-to-back order of the layers is sprite layer 1, background 1, sprite layer 2, background 2, sprite layer 3, background 3, background 4.

Within a layer the compositing rule of Section 5.7 applies: a pixel with value 0 is transparent. A tile whose priority bit is set is in front of everything, sprite layer 1 included; its pixels of value 0 are still transparent.

6.5 Fetch Schedule

The device reads video memory once per PPU cycle, and draws four pixels in eight PPU cycles, so every group of four pixels has eight fetch slots. Mode 1 uses them as follows:

Slot	Fetch
0	Background 1 tilemap entry
1	Background 2 tilemap entry
2	Background 3 tilemap entry
3	Background 4 tilemap entry
4	Background 1 tile data word
5	Background 2 tile data word
6	Background 3 tile data word
7	Background 4 tile data word

Table 6.1: Mode 1 Fetch Slots per Group of Four Pixels

An 8 by 8 tile is eight pixels wide, so the tilemap entries are fetched twice for each tile, once for each group of four, and the device does not keep an entry for the second group. The fetched pixels are combined into four finished pixels at the start of the next group (Chapter 10).

All eight slots are in use, so Mode 1 leaves none for sprite tile data; sprites are fetched in the blanking intervals (Chapter 9). A background with 2-bit tiles reads its tile data word only in every second group, and its slot is idle in the others.

CHAPTER 7

Mode 2: Isometric Backgrounds

7.1 Overview

Mode 2 draws two backgrounds as a 2:1 isometric ground plane. Each background is an ordinary map of square tiles, but the device does not draw the map in rows and columns. It samples the map along diagonals so that every square tile appears as a diamond, 32 pixels wide and 16 pixels high, and the whole map appears as the flat, tilted landscape of a strategy or simulation game.

Selected by: `MODE` = 1 in the Base Configuration register.

Layers: Two isometric backgrounds, 1 (front) and 2 (back), and an orthogonal status layer, background 3. Sprites are drawn as in Mode 1 (Chapter 9).

Tiles: 16 by 16 texels, 4 bits per texel, 64 words each, in the staircase layout of Section 7.4.

Map: One 32 by 32 tilemap per background, wrapping in both directions.

7.1.1 What the Mode Is For

Mode 2 is a specialised mode. It is good at one picture, a large flat map seen from above at an angle, with pixel-smooth scrolling and a map that is the game's own grid. Transport and city-building games are of this kind.

It is poor at nearly everything else, and the reasons come from the same design decision that makes it good at its job:

- Every screen pixel belongs to exactly one tile of a layer. A tile cannot overhang its neighbours the way a sprite can, so buildings, trees and vehicles are not drawn by the tile hardware. They are sliced into many tiles by a tool (Section 7.10) or drawn by sprites.
- Each group of four pixels needs a word of tile data, and sometimes two, because neighbouring pixels sit on different rows of the tile and the group can lie on the edge of two tiles. That keeps most of the device's fetch bandwidth in use, so the mode has two isometric backgrounds instead of three and no slot reserved for sprites (Section 7.5).
- The view cannot be rotated by a register. Art is drawn for one facing.

Programs that want side-scrolling backgrounds, parallax or many moving objects belong in Mode 1.

7.2 Geometry

Mode 2 works in a *plane*: the screen pixel (x, y) of a background plus that background's scroll, so the plane pixel is $(X, Y) = (x + \mathrm{SCRX},\ y + \mathrm{SCRY})$. Plane pixel (X, Y) shows the map texel (u, v) given by

$$u = Y + \lfloor X/2 \rfloor,$$
$$v = Y - \lfloor (X+1)/2 \rfloor,$$

and the texel belongs to the tilemap cell $(c_u, c_v) = (\lfloor u/16 \rfloor, \lfloor v/16 \rfloor)$, with both indices taken modulo 32, at position $(u \bmod 16, v \bmod 16)$ inside its tile. The formula uses two adders and no multiplier.

One step to the right moves half a texel along u and half a texel back along v; one step down moves one texel along each.

Three properties of the mapping matter:

- $u - v = X$ for every pixel. This is what makes four neighbouring pixels a staircase of four texels (Section 7.4).
- It is a bijection. Every texel of a tile is shown by exactly one pixel, so a tile has as many pixels as texels, 256, and nothing is lost or repeated when an orthogonal tile is drawn as a diamond. An artist's diamond can be unwrapped into a square tile and shown again without any change (Section 7.10).
- The picture is a 2:1 isometric (dimetric) view. It is not a 45 degree rotation of the tile, which would need a multiplication by 0.707 and would not be a bijection.

7.2.1 The Diamond Footprint

The pixels that show one tile form a diamond. Its top pixel is the plane pixel

$$(X, Y) = (16(c_u - c_v),\ 8(c_u + c_v)),$$

which shows texel $(0, 0)$ of the tile. Table 7.1 gives the rows of the footprint; each is centred on the column of the top pixel. The diamond is 31 pixels wide at its widest and 16 pixels high, which is why the lattice is described as a 32 by 16 diamond: the neighbouring cell $(c_u + 1, c_v)$ has its top pixel 16 pixels to the right and 8 pixels down, and the cell $(c_u, c_v + 1)$ has it 16 pixels to the left and 8 pixels down.

Row	Half-width	Width	Row	Half-width	Width
0	0	1	8	15	31
1	2	5	9	13	27
2	4	9	10	11	23
3	6	13	11	9	19
4	8	17	12	7	15
5	10	21	13	5	11
6	12	25	14	3	7
7	14	29	15	1	3

Table 7.1: Rows of a Tile's Diamond Footprint (256 Pixels in All)

The footprint is not quite symmetric from top to bottom: the top row is one pixel and the bottom row is three. This is the shape that makes the mapping a bijection, and it is the shape the diamonds of a map tessellate in. A visible area of 256 by 224 pixels shows 224 tiles' worth of pixels.

7.3 Tilemap, Scrolling and Wrapping

The tilemap is 32 by 32 cells by default, or 64 cells across or down (Section 5.5); the entry for cell (c_u, c_v) is at the address that section gives. The entry fields are those of Section 5.4; in Mode 2 the tile number and the palette selection are used, the flip bits darken and lighten the tile (Section 7.8), and the priority bit is ignored.

The scroll registers move the plane under the screen in whole pixels: increasing `SCRX` moves the picture left by a pixel, and increasing `SCRY` moves it up by a pixel. Scrolling is in screen pixels, not in map texels, so a background can be positioned to any pixel and moved smoothly in any

direction. The registers are sampled once per line, 32 master clocks before the first visible pixel (Section 7.5), so a change made during a line takes effect on the next.

With both scrolls at 0, the top left pixel of the screen is the top pixel of cell $(0, 0)$. The map wraps: a map W cells across wraps its texel coordinates u at $16W$ and a map H cells high wraps v at $16H$. For the default 32 by 32 map that is 512, and the plane repeats every 1024 pixels horizontally and every 512 vertically. At the upper right of the screen v is negative, so that region shows the bottom rows of the tilemap.

7.3.1 Moving Around the Map

The scroll registers move the picture across the screen, but the map's own directions, u and v, run diagonally on the screen: u runs down and to the right, and v runs down and to the left. To move the view by $(\Delta u, \Delta v)$ texels along the map, change the scroll registers by

$$\Delta\mathrm{SCRX} = \Delta u - \Delta v,$$
$$\Delta\mathrm{SCRY} = (\Delta u + \Delta v)/2.$$

Moving along u by two texels is therefore two pixels in SCRX and one in SCRY; moving along v by two is -2 and $+1$. Moving straight across the screen is $\Delta u = -\Delta v$, and moving straight down is $\Delta u = \Delta v$. A program that keeps its position in map texels converts it once a frame, in vertical blanking.

The scroll registers hold 0 to 1023, so a program keeps them modulo 1024. When SCRX is brought back by 1024 (from 1024 or more to 0 or more) it must also bring SCRY back by 512, whatever the size of the map, so that the picture does not jump; with both done the picture is as it was, and the whole of the map can be reached.

For example, a view at $(\mathrm{SCRX}, \mathrm{SCRY}) = (100, 50)$ that is to move 6 texels along u and 2 along v needs $\Delta\mathrm{SCRX} = 6 - 2 = 4$ and $\Delta\mathrm{SCRY} = (6 + 2)/2 = 4$, so the new scroll is $(104, 54)$. A view at $(1020, 700)$ that moves 10 texels along u needs $+10$ and $+5$, which would be $(1030, 705)$; the program writes $(1030 - 1024,\ 705 - 512) = (6, 193)$ instead, and the picture is the same.

The tilemap is at most 64 by 64 cells, 1024 by 1024 texels. A map larger than that is kept in memory by the program, which rewrites the rows and columns of the tilemap that have moved out of sight with those that are coming into view. Vertical blanking is the time to do it (Section 3.3.1); a program that scrolls slowly needs only a few entries a frame.

7.4 The Staircase Tile Format

The device reads one 16-bit word, which holds four pixels, for every four pixels it draws. In Mode 1 those four pixels are side by side in one row of a tile. In Mode 2 they are not: moving one pixel right moves half a texel along u and half a texel back along v, so four pixels starting at a plane column X_0 that is a multiple of four sample the staircase

$$(a, b),\ (a, b - 1),\ (a + 1, b - 1),\ (a + 1, b - 2)$$

of four texels, where (a, b) is the texel of the first pixel. These staircases partition the plane. The tile format stores each one in a word, so that the four pixels of a group come from a single read.

7.4.1 Locating a Texel

Texel (t_u, t_v) of a tile, each coordinate from 0 to 15, is stored in word

$$w = 16 \left\lfloor \frac{(a - b) \bmod 16}{4} \right\rfloor + (a \bmod 16)$$

of the tile, in nibble i, where

$$i = (t_u - t_v) \bmod 4,$$
$$(a, b) = \big(t_u - \lfloor i/2 \rfloor,\ t_v + [0, 1, 1, 2]_i\big).$$

Nibble 0 is bits 15–12, the leftmost pixel, and nibble 3 is bits 3–0, as in Figure 5.2. For example, texel $(5, 3)$ has $i = 2$, so $(a, b) = (4, 4)$, which is word 4, nibble 2.

7.4.2 Why It Works

The tile is treated as if it wrapped around on itself, so a staircase that leaves the right or bottom edge continues at the left or top edge of the same tile. With that rule the tile has exactly 64 staircases, one for each pair of $a \bmod 16$ and $(a - b) \bmod 16$ that is a multiple of four, and every texel is in exactly one of them. The 256 texels fill the 64 words with no gaps, so a Mode 2 tile costs the same 64 words as a 16 by 16 tile in any other layout.

When the four pixels of a group fall in two different tiles of the map, the staircase has two parts. Each part is in the same word number of the two tiles, and the device reads that word from both tiles and takes each pixel's nibble from the tile the pixel falls in. A group never touches more than two tiles, and the word number is the same for both. About 19% of groups are split in this way (Section 7.5).

The device never rearranges tile data. A tool that prepares tiles (Section 7.10) writes the format, and the CPU copies it to video memory unchanged.

7.5 Fetch Schedule and Pipeline

7.5.1 Slots

As in Mode 1, a group of four pixels takes eight PPU cycles, and every cycle is one read of video memory. Mode 2 gives each of its two backgrounds four of them:

Slot	Fetch	Used
0	Background 1 tilemap entry, cell A	Always
1	Background 1 tile data word, tile A	Always
2	Background 1 tilemap entry, cell B	Only when the group has two cells
3	Background 1 tile data word, tile B	Only when the group has two cells
4–7	The same four fetches for background 2	

Table 7.2: Mode 2 Fetch Slots per Group of Four Pixels

Cell A is the cell of the group's first pixel. Cell B is the other cell, when one of the remaining pixels falls in a different cell; otherwise the B slots are idle. Over any 16,384 groups, 3072 of them (18.75%) have a second cell. That is geometry, not a property of the tile data: the edges between cells run diagonally, move two pixels sideways on each line, and so fall inside a group of four about three times in four.

On average a background uses 2.375 of its four slots and the two together use 4.75 of the eight. The worst case uses all eight.

7.5.2 Tilemap Entry Fetches

A cell is 16 pixels wide or more, but the device reads the tilemap entry of its cell afresh for every group, and does not keep the entry it read for the previous group.

7.5.3 Spare Slots

Which slots are spare depends only on the scroll positions, not on the tile data, because the cells a group touches depend only on position. Over all the scroll positions of the two backgrounds, 192 to 224 of the 512 slots of a line are spare, 208 on average. These slots feed the status layer through a queue of words fetched ahead of when they are needed (Section 7.6).

7.5.4 Groups and Fine Scrolling

Groups are aligned to the plane, not to the screen: the first group a background fetches for a line starts at plane column $4\lfloor \mathtt{SCRX}/4 \rfloor$. Each background holds a window of two groups. When SCRX is not a multiple of four, the line's first pixel is at position $\mathtt{SCRX} \bmod 4$ in the window, and the window advances by four pixels at each group. Fine horizontal scrolling therefore costs no extra fetches. The vertical scroll adds to the line number and needs nothing.

7.5.5 Per-Line Sequence

1. 32 master clocks (16 PPU cycles) before the first visible pixel, in the back porch, the device samples the scroll registers of both backgrounds, and then fetches two groups for each: the first, then the second.

2. From the first visible pixel on, the device shows the pixels of one group of four, and at the same time fetches the group two groups ahead of it in the plane.

3. The tilemap base, the tile data base and the palette configuration are read live with every fetch. They must not change during the visible part of a line.

The prefetch needs 16 PPU cycles of the 130 cycles before the picture (Chapter 10). The device draws no lines in vertical blanking and does not fetch for them.

7.6 The Status Layer

Background 3 in Mode 2 is an orthogonal layer for a status display: ordinary 8 by 8 tiles in rows and columns, as in Mode 1, not isometric. The tile size bit B3S is ignored. It is meant for text, bars and windows:

- It has the tilemap and tile data of Mode 1: a 32 by 32 tilemap at B3MAP, and tiles of 16 words at B3TILES, 2 words for each row, 4 bits per pixel (Section 5.3.1), or with T2 set in B3PAL tiles of 8 words of 2 bits per pixel (Section 5.3.5). The tilemap entry's tile number and palette selection are used, and B3PAL applies. Flips are ignored.

- It does not scroll: the scroll registers B3SCRX and B3SCRY are ignored, and screen pixel (x, y) shows tile $(\lfloor x/8 \rfloor, \lfloor y/8 \rfloor)$.

- The priority bit of the tilemap entry puts a tile in front of everything, including sprite layer 1. A tile without it is behind everything, as the last background is in Mode 1. The status layer's pixels of value 0 are transparent.

- B3D in the Base Configuration register turns it off. A program that does not use it must set B3D, or the device reads whatever B3MAP and B3TILES point at.

7.6.1 The Queue

The status layer has no fetch slots of its own. It uses a slot when the isometric layers do not need it, and because those slots are not spread evenly, it keeps a queue of the words it has fetched ahead of the picture. In each group the device shows the oldest queued group, and in each spare slot it fetches the next tilemap entry (for every second group) or tile word for a group up to four groups ahead of the picture. Before the first visible pixel, in the back porch, it fills the queue with the first four groups, which takes six reads.

The queue is four groups deep because that is what the worst case needs. Over all 16,384 pairs of scroll positions of the two isometric backgrounds, the depth needed is two groups for 14,464 pairs, three for 1,872 and four for 48. A shallower queue lets the layer run short of data, and the group is then shown transparent; the depth is a hard requirement.

The queue is the first place in the device where the fetch of a layer depends on what the other layers are doing, so it needs a small scheduler: in each cycle, if the slot is not needed by an isometric layer and the queue has room, read for the status layer. It also assumes that the CPU does not read or write video memory in the visible part of a line (Section 3.3.1).

7.7 Two-Bit Tiles

An isometric background can have 2-bit tiles, to halve the video memory its tiles take. Terrain and roads need few colours; buildings keep 4 bits. Bit 2 of the layer's palette configuration register, `T2`, selects them, for background 1 and background 2 separately. The orthogonal backgrounds have the same bit (Section 5.3.5).

- Groups are eight pixels, aligned to the plane, and the word holding the eight texels of a staircase is one 16-bit word of eight 2-bit pixels, the leftmost in bits 15–14. A tile is 32 words, not 64, at tile data base plus 32 times the tile number. The 32 staircases of a tile cover its 256 texels exactly once.
- The word's number is $16\lfloor(x_0 \bmod 16)/8\rfloor + (u_0 \bmod 16)$, where x_0 is the plane column of the group's first pixel and u_0 the u of that pixel, the same construction as in Section 7.4. As for 4-bit tiles, both cells of a group that straddles two use the same word number.
- A group of eight never touches more than two cells. 1,792 of 4,096 groups, 43.75%, touch two, against 18.75% for groups of four.
- Value 0 is transparent, so a pixel has three colours. The palette rule is that of 4-bit tiles (Section 5.6), with values 0 to 3; `SIZE` should be 0 (four colours).
- The layer fetches a group of eight for every second group of four pixels, and the other groups leave all four of its slots spare. A line then needs 80 to 96 reads for the layer, 92.0 on average, against 144 to 160, 152.0 on average, for a 4-bit one, not counting the prefetch. The first group of a line can be the second half of a staircase, so the first group is fetched in the back porch whole.
- A tilemap in which the demonstration ground of Figure 7.2 is drawn in three colours has 14 unique tiles and takes 448 words as 2-bit tiles, against 896 as 4-bit ones. Flips stay unsupported.

7.8 Shading

An isometric layer cannot flip its tiles, so the two flip bits of its tilemap entries are used for something else:

V (bit 15)	H (bit 14)	Effect on the tile's pixels
0	0	Normal
1	0	Darkened: each colour channel is halved
0	1	Lightened: each channel moves halfway to its maximum, 31
1	1	Normal

Table 7.3: Shading Bits of an Isometric Tilemap Entry

Shading is applied to the colour after the palette lookup, so one tile serves for a lit and a shaded version of the same ground. It costs a shift and an add per colour channel at the output. A program can darken the cells it has not yet explored (fog of war), the cells under a tree or at night, and

lighten highlighted ones, without extra tiles or palettes. Shading applies in Mode 2 and in Mode 3's isometric background. The orthogonal backgrounds of Mode 3 read the same two bits as flips (Section 5.4); the status layer ignores them.

A sprite can darken as well. When the `SHD` bit of the Base Configuration register is set, a sprite pixel of value 15 in sprite palette 7 draws nothing; instead it darkens whatever lies behind the sprite's layer (Chapter 9). A sprite on layer S2, for example, darkens the isometric background beneath it but not tiles of background 1, which are in front of that layer. Darkening does not accumulate: a tile that is already darkened is not darkened further by a shadow, and a darkened pixel that is lightened is normal.

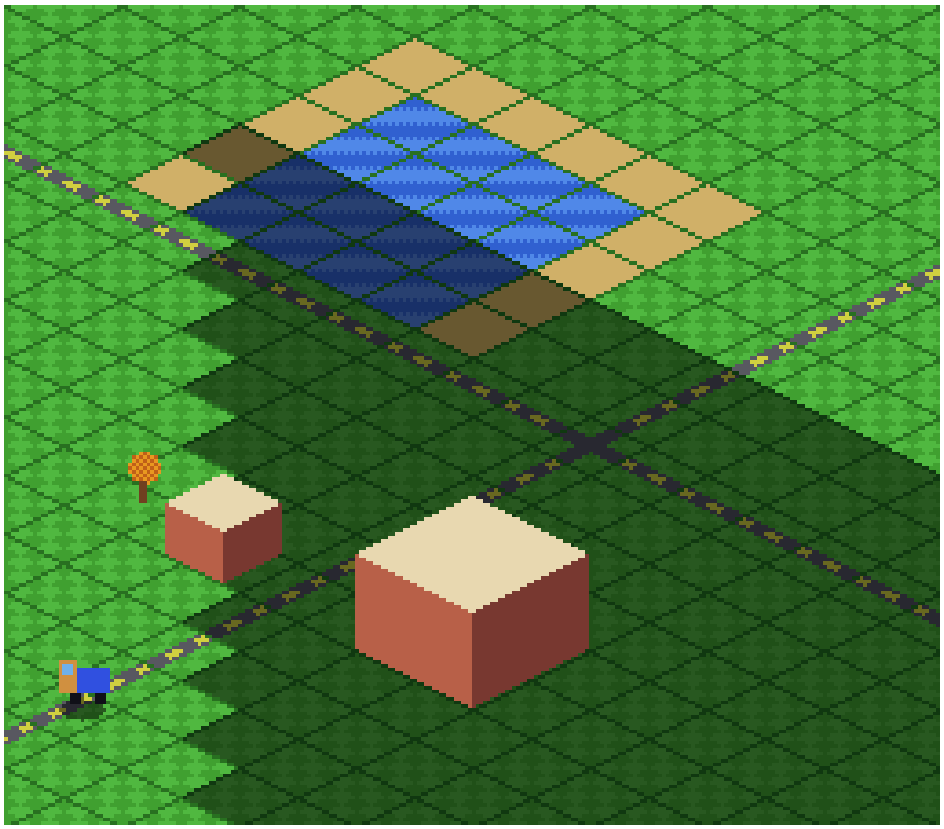


Figure 7.1: Fog of War and a Vehicle Shadow

7.9 Layers, Priority and Height

Background 1 is in front of background 2. A pixel of background 1 whose value is not 0 hides the pixel of background 2 at the same screen position, and if both are 0 the device shows palette entry 0 (Section 5.7). The disable bits `B1D` and `B2D` of the Base Configuration register make the corresponding background transparent. The two backgrounds have their own tilemap, tile data, scroll and palette registers; they share nothing but the palette. Background 4 is not drawn in Mode 2.

Because each background scrolls independently, the two can be positioned relative to each other by any amount. That is what a height offset is: a background whose `SCRY` is h smaller than another's is drawn h pixels lower on the screen, and one whose `SCRY` is larger is drawn higher. The scroll is not part of the tile data: a program scrolls a layer freely and the whole picture moves with it. What matters is the offset between the two layers. The tiles of a layer are cut along its own grid, and a layer with a different scroll has its grid in a different place on the screen, so the tool that slices the artist's picture is told the scroll each layer is drawn at while the picture is cut (Section 7.10), and the program keeps the offset between the layers after that. Offset layers are for one purpose: a tall object overhangs the cells behind it, and in one layer it must share those cells with whatever else is

there, so objects that would otherwise conflict are put on a layer with a different grid. Two layers cannot make every overhang work. Anything that moves is a sprite, or the program rewrites tiles as it moves.

7.10 Authoring Tiles

An artist draws in screen space: a picture of diamonds, roads, water and buildings, as it should appear. A tool turns that picture into tilemaps and tiles with this algorithm:

1. Choose the scroll $(\texttt{SCRX}, \texttt{SCRY})$ at which the picture is drawn. It only decides where the picture falls on the layer's grid; the program may scroll the layer anywhere afterwards. A layer that is to sit offset from another has its own scroll here, and the program keeps that difference.
2. For each pixel (x, y) of the picture, find the texel (u, v) shown by the plane pixel $(x + \texttt{SCRX},\ y + \texttt{SCRY})$ and the cell and the position within the tile (Section 7.2), and store the pixel's value there.
3. Texels that no pixel reaches stay 0, transparent.
4. Give each different tile image a tile number, and reuse the number for identical images. Tile 0 is the empty tile.
5. Pack each tile into the staircase layout and write the tilemap entry of each cell.

Because the mapping is a bijection, the result shows the picture again pixel for pixel. A picture of ground, roads, a lake and two buildings, drawn in screen space and sliced this way, is shown with no differing pixel.

The tiles of a picture drawn on a regular lattice repeat, and step 4 saves their storage: the ground of the example picture, a 256 by 224 screen of grass, two roads and a lake, takes 15 tiles, counting the empty tile 0. The two buildings of the example, whose walls overhang many cells, take 28, again counting the empty tile, and only because they were drawn once and cut up by the tool. An artist does not need to draw cell by cell.

7.11 Restrictions

Mode 2 has these restrictions, all of which are consequences of the design above:

- Two isometric backgrounds and an orthogonal status layer. The status layer does not scroll, and its flip bits are ignored.
- 16 by 16 texel tiles only. The tile size bits of the `TILESIZE` register are ignored. An 8 by 8 texel tile would be a 16 by 8 diamond; it is not supported.
- A tilemap of 32 or 64 cells across and down (Section 5.5).
- Tiles cannot be flipped: the flip bits darken and lighten instead (Section 7.8). A mirrored tile is a different tile, and the tool writes it.
- The priority bit is ignored.
- No fetch slots for sprites in the visible part of a line.
- 4 or 2 bits per texel, chosen for each isometric background (Section 7.7).
- No rotation of the view. Art is drawn for one facing, and a program that shows several facings keeps a set of tiles for each.
- Heights are not part of the hardware. A raised terrain is a background with a different scroll (Section 7.9).

7.12 A Worked Example

Figure 7.2 shows the picture the device draws for an example scene. The ground, on background 2, is a map of grass, two crossing roads, and a lake with a sandy shore. The buildings, on background 1, are a tower of two cells by two and a house of one cell. Background 1 is scrolled down by 8 pixels, and its picture was sliced against the grid that results; the picture on the screen is the same however it is sliced.

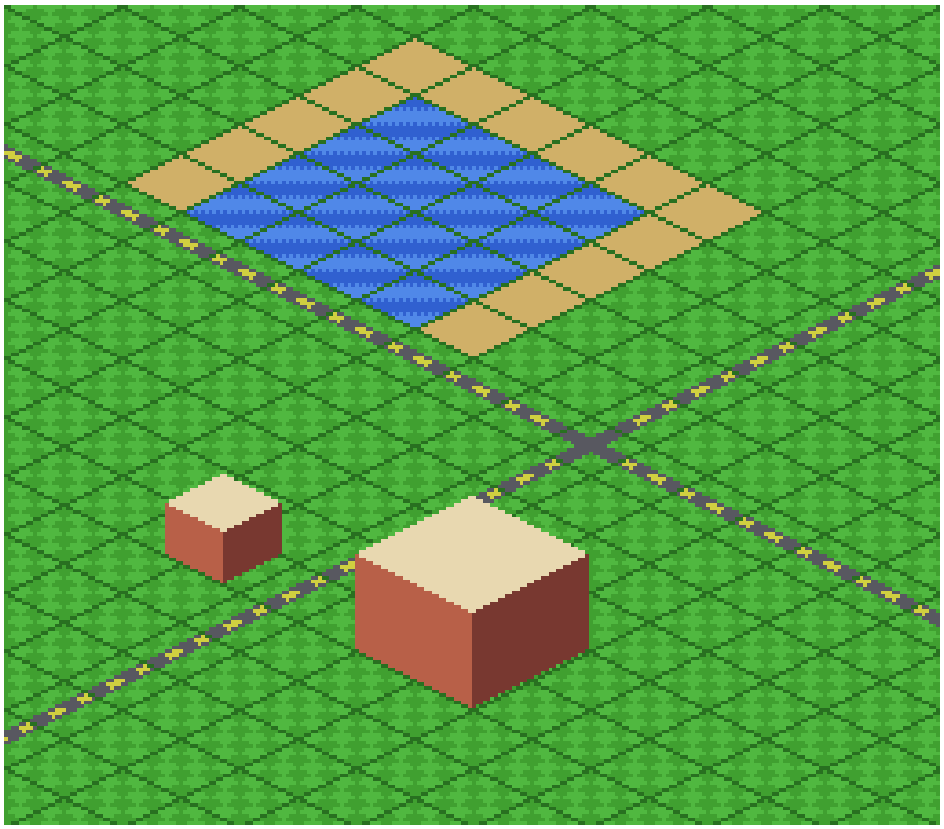


Figure 7.2: Mode 2 Output for the Example Scene

The registers were set as follows:

Register	Background 1 (buildings)	Background 2 (ground)
`CONFIG`	`MODE` = 1, `B3D` = 1	
Tilemap address	`0x8000`	`0x8400`
Tile data address	`0x9000`	`0xA000`
Scroll X, scroll Y	0, 8	0, 0
Palette configuration	`OFFSET` 0x10, 16 colours	`OFFSET` 0x00, 16 colours
Palette entries used	0x11–0x13	1–8

Table 7.4: Register Values of the Example Scene

7.13 Programming Mode 2

A program uses Mode 2 as follows:

1. During vertical blanking, load the palette and write the tile data and the tilemaps into video memory.

2. Write the tilemap and tile data address registers and the palette configuration registers of backgrounds 1 and 2.
3. Write the Base Configuration register with `MODE` = 1.
4. Each frame, during vertical blanking, write the scroll registers. The scrolls of the two backgrounds are independent.

To leave Mode 2, write `MODE` = 0 during vertical blanking. The address and palette registers keep their values, so Mode 1 can reuse them.

CHAPTER 8

Mode 3: Orthogonal Backgrounds over an Isometric One

8.1 Overview

Mode 3 keeps one isometric background and gives the fetch slots of the other to two orthogonal ones. It is for a game with an isometric ground and a user interface that needs more than the status layer of Mode 2: a status display and a layer of windows over it, with the map, and the vehicles and buildings as sprites, underneath.

Selected by: `MODE` = 2 in the Base Configuration register.

Layers: Orthogonal backgrounds 1 and 2 in front, and an isometric background 3 behind them. The order front to back is S1, B1, S2, B2, S3, B3, as in Mode 1 without background 4 (Section 6.4).

Registers: Each background uses its own tilemap, tile data, scroll and palette configuration registers, as in Mode 1. Background 3 is the isometric one, so `B3MAP`, `B3TILES`, `B3SCRX`, `B3SCRY` and `B3PAL` follow Mode 2 (Chapter 7).

Mode 3 adds no hardware to Mode 2. Each background is the same fetch unit as in the other modes, and the mode only says which slots of a group of four pixels belong to which background. The price is the status layer of Mode 2: with no spare slots there is no queue, and no second isometric background.

8.2 The Orthogonal Backgrounds

An orthogonal background is the background of Mode 1 (Chapter 6): a tilemap of 8 by 8 or 16 by 16 tiles, 32 by 32 entries or larger (Section 5.5), that wraps in both directions. Its tilemap entry holds a tile number and a palette selection (Section 5.4), and its flip bits and priority bit apply as they do in Mode 1. Tiles are 4 bits per pixel, 16 words each (Section 5.3.1), or, when `T2` is set in its palette configuration register, 2 bits per pixel, 8 words each (Section 5.3.5). The palette is chosen as in Section 5.6, with the register's own `OFFSET`. Pixels of value 0 are transparent. The priority bit puts a tile in front of every sprite layer.

8.2.1 Scrolling

Each orthogonal background scrolls independently. Its scroll registers are sampled once per line, as in Mode 2. The device fetches groups of four pixels aligned to the plane, so that group g of the line is the plane group $\lfloor \mathrm{SCRX}/4 \rfloor + g$. For tiles w pixels square (8 or 16), a group of four lies within one tile, in tile column $\lfloor 4q/w \rfloor \bmod C$ and tile row $\lfloor (\mathrm{line} + \mathrm{SCRY})/w \rfloor \bmod R$, where C and R are the width and height of the tilemap in cells (32 or 64, Section 5.5). The row of pixels within the tile is $(\mathrm{line} + \mathrm{SCRY}) \bmod w$, and the plane wraps after wC pixels across and wR pixels down. When `SCRX` is not a multiple of four, the line's first pixel is at position $\mathrm{SCRX} \bmod 4$ in a window of two groups, as in Section 7.5. Fine scrolling costs no extra fetches.

8.3 Fetch Schedule

The eight slots of a group of four pixels have fixed owners:

Slot	Fetch	Used
0	Background 1 tilemap entry	Always (2-bit tiles: every second group)
1	Background 1 tile data word	Always (2-bit tiles: every second group)
2, 3	The same two fetches for background 2	
4–7	The four fetches of an isometric background, for background 3	As in Table 7.2

Table 8.1: Mode 3 Fetch Slots per Group of Four Pixels

The orthogonal backgrounds use four slots in every group, and the isometric background uses two to four, so a group uses six or more of the eight and the worst case uses all eight. No slot is left for anything else in the visible part of a line. Sprites are fetched in the horizontal blanking interval, as in the other modes (Chapter 9), and the back-porch prefetch of two groups is the same as in Mode 2.

Because no background depends on a slot that another leaves spare, the schedule is fixed and needs no queue.

8.4 Differences from Mode 2

- There is no status layer: the orthogonal backgrounds take its place, and `B3D` disables the isometric background.
- `B1D`, `B2D` and `B3D` disable the corresponding background, as in Mode 2. Background 4 is not drawn in Mode 3.
- The isometric background is background 3, so it is behind the orthogonal ones. The shading bits of its tilemap entries (Section 7.8) apply to it; the orthogonal backgrounds read the same two bits as horizontal and vertical flips (Section 5.4).
- Only one background is isometric, so the height offsets of Section 7.9 between two isometric backgrounds are not available. Buildings and vehicles are sprites.

8.5 Restrictions

- One isometric background, with the restrictions of Mode 2 (Section 7.11).

8.6 Programming Mode 3

1. Write the tilemap, tile data and palettes of the three backgrounds. The orthogonal tiles use the format of Section 5.3.1 or 5.3.5; the isometric tiles use the staircase format (Section 7.4) or its 2-bit form.
2. Set `B1MAP`–`B3MAP`, `B1TILES`–`B3TILES` and the palette and scroll registers.
3. Write 2 to the `MODE` field. Do it in vertical blanking: a change during a line takes effect at the start of the next one.

CHAPTER 9

Sprites

9.1 Overview

The device draws three layers of sprites, S1 to S3, each 4 bits per pixel. The sprites are described by a table of 128 entries of 32 bits each, mapped at offsets `0x7000`–`0x70FF` of the device segment, and their tile data is at the address in the `STILES` register. A sprite layer sits between background layers in the front-to-back order S1, B1, S2, B2, S3, B3, B4. Sprite tiles are 8 by 8 or 16 by 16 pixels, chosen for each sprite layer by the `TILESIZE` register.

9.2 Sprite Table Entry

Each entry is two words. The table is internal memory of the device: reading it costs no video memory bandwidth, which is what lets the device scan all 128 entries while it draws the previous line. The contents of the table are undefined at reset. A sprite whose layer field is 3 is hidden.

Word	Bits	Field
0	15	`V`: flip vertically
0	14	`H`: flip horizontally
0	13–11	`P`: sprite palette 0–7
0	10–9	`L`: sprite layer, 0–2 for S1–S3; 3 hides the sprite
0	8–0	`X`: left edge, a signed 9-bit number (-256 to 255)
1	15–8	`Y`: top line, 0–255, wrapping, so a sprite can enter from above
1	7–0	`T`: tile number

Table 9.1: Sprite Table Entry

The tile number is 8 bits because the entry has no room for more: 32 bits cannot hold a 9-bit X, an 8-bit Y, three flags, a 3-bit palette and a 2-bit layer together with a 10-bit tile number, as a background tilemap entry has. A sheet of 256 tiles is 4,096 words of 8 by 8 sprites, or 16,384 words (half of video memory) of 16 by 16 sprites. A program that needs more switches the `STILES` register between frames.

9.3 Sprite Tile Data

A sprite tile is stored as a background tile is (Sections 5.3.1 and 5.3.3), 4 bits per pixel, leftmost pixel in bits 15–12 of a word. An 8 by 8 tile is 16 words, two for each row. A 16 by 16 tile is four 8 by 8 quadrants in the order top left, top right, bottom left, bottom right, 64 words in all. The word for pixel column c and row r of tile t is at

$$\text{STILES} + t \cdot (s^2/4) + 16q + 2(r \bmod 8) + \lfloor (c \bmod 8)/4 \rfloor,$$

where s is the sprite size and the quadrant q is 0 for an 8 by 8 tile and $2\lfloor r/8 \rfloor + \lfloor c/8 \rfloor$ for a 16 by 16 one. A row of a 16 by 16 sprite is therefore two words from the left quadrant and two from the right.

Value 0 is transparent. A pixel of value v in a sprite of palette p selects palette entry $\texttt{0x80} + 16p + v$, so the sprites own the upper half of the palette: eight palettes of 16 entries, which is independent of the background palette offsets. The exception is value 15 of palette 7 when `SHD` is set, the shadow colour (Section 9.4).

9.4 Shadow Pixels

When the `SHD` bit of the Base Configuration register is set, a sprite pixel of value 15 in sprite palette 7 is a shadow. With the bit clear, which it is at reset, it is an ordinary colour. A shadow draws nothing itself; it darkens what is behind its sprite layer, as the shading of Section 7.8 does (each colour channel halved). It is for the shadow under a vehicle or a building. It works in every mode. While `SHD` is set, the palette entry `0xFF` is never used.

The line buffer holds one sprite pixel per column, so the rules for shadows are simple: a shadow pixel never covers a drawn sprite pixel, and a drawn sprite pixel always replaces a shadow pixel, whatever their layers. A shadow therefore darkens the backgrounds, not other sprites, and darkening does not accumulate.

9.5 Drawing a Line

The device draws sprites with a line buffer of 256 entries, which it fills before the line is shown:

1. **Scan.** During the visible part of the previous line the device walks the table and keeps the first 32 sprites, in table order, whose rows cross the next line and whose layer is neither hidden nor disabled in the Base Configuration register. A sprite is on the line when $(\text{line} - Y) \bmod 256$ is less than its size. Further sprites on the line are dropped.

2. **Fetch.** In the horizontal blanking interval it reads one row of tile data for each kept sprite, one word per PPU cycle. A word with no pixel on the screen is not read.

3. **Paint.** Each word is expanded into the line buffer. Where sprites overlap, the one on the lower layer number wins, then the one with the lower table index. The buffer holds one pixel per column, with its palette entry and its layer.

Keeping only the frontmost sprite pixel is enough. A sprite pixel that lies behind another sprite pixel could never be the one shown: whatever would cover it, the frontmost sprite pixel is in front of the same things.

When the line is shown, the compositor places the buffered sprite pixel among the backgrounds in the order S1, B1, S2, B2, S3, B3 (and, in Mode 1, B4 last), taking the first opaque pixel. This is the same rule in both modes; Mode 2 simply has no third background.

Line 0 is always blank (Section 10.3.1), so it has no sprites and needs none fetched: the device scans and fetches during it for line 1. A sprite with $Y = 0$ therefore loses its first row, and a program that wants a sprite whole gives it $Y \geq 1$.

9.6 How Many Sprites

9.6.1 Fetch Budget

The horizontal blanking interval is 340 master clocks, 170 PPU cycles, and neither mode fetches background data in it except for a prefetch of 22 cycles before the first pixel (16 for the two isometric backgrounds, six for the status layer). That leaves 148 video memory reads per line for sprites, whatever the mode. A sprite costs one read per word of its row.

Size	Reads per sprite	Fit in 148 reads	Reads for 32 sprites
8 by 8	2	74	64
16 by 16	4	37	128

Table 9.2: Sprite Fetch Cost per Line, Blanking Only

The time budget is therefore not the limit; 32 sprites of either size fit with room to spare. The limit is the capacity of the line registers that hold a kept sprite between the scan and the fetch. The device keeps 32, and one sprite is about 40 bits of state, so 32 is some 160 bytes of registers. A program can use all 128 entries in a frame, but no more than 32 on any one line; a line that has more drops the highest-numbered ones, and sets the `SOV` bit of the Status register (Chapter 4).

9.6.2 Using the Visible Slots as Well

The fetch slots of the visible part of the line could be used as well, but the sprites for the next line could then no longer be painted into the buffer that is being shown, so a second line buffer (another 256 entries) is needed.

Mode 1 has no slot to spare: its four backgrounds use all eight in every group of four pixels.

Mode 2 has no slot reserved, but some of its slots are spare. Which ones depends only on the scroll position, not on the tile data (Section 7.5.3), so the device can know in advance. Over all scroll positions 192 to 224 slots per line out of 512 are spare, 208 on average, and the status layer takes some of them.

The spare slots of Mode 2 are not needed to reach the line limit, so the device does not use them.

9.7 Sprites in Modes 2 and 3

Sprites are drawn the same way in Modes 2 and 3, at screen positions. In Mode 2, which has two isometric backgrounds, the three sprite layers sit as S1, B1, S2, B2, S3. In Mode 3 the order is the same as in Mode 1 without background 4, S1, B1, S2, B2, S3, B3, with the isometric background last. A program sorts its sprites against the map itself: a lorry that is to pass behind a tower on B1 goes on S2, one that is to pass in front goes on S1, and it changes layer as it crosses. Figure 9.1 shows both: the lorry on S1 is in front of the tower, and the one on S2 would be hidden by it.

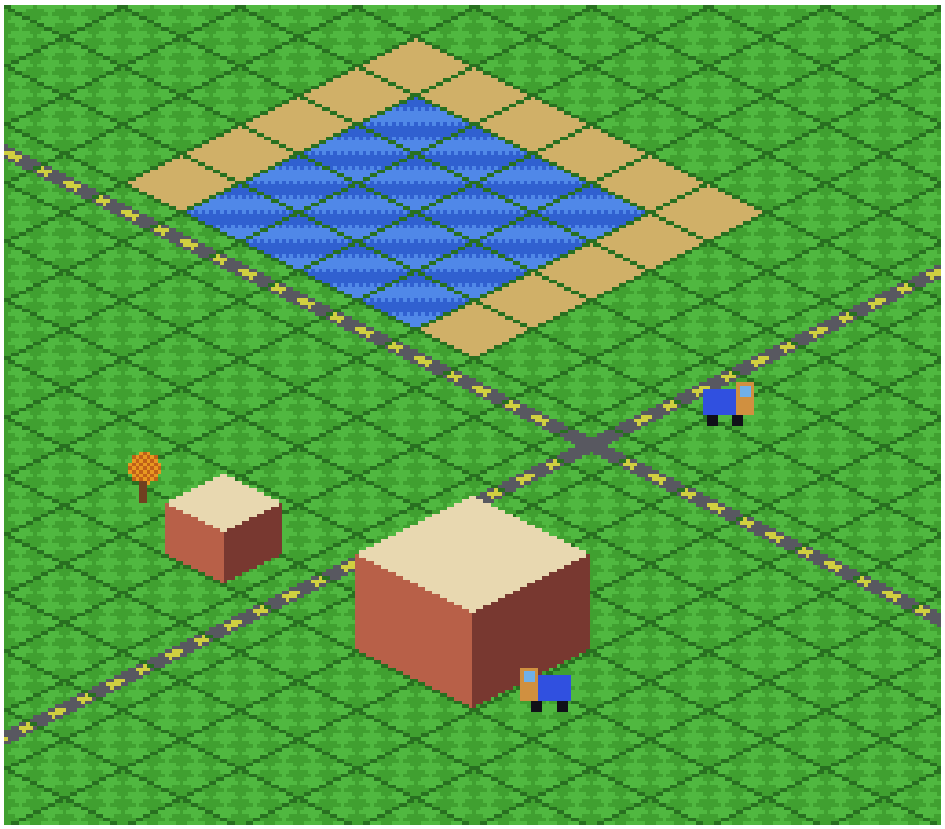


Figure 9.1: Two Lorries and a Tree over the Mode 2 Demonstration Scene

9.8 Programming Sprites

1. Write the sprite tile data into video memory, at the address that will be written to `STILES`.
2. Write `STILES`, and the sprite tile size bits of the `TILESIZE` register.
3. Write the table: entry n is the words at offsets `0x7000` $+2n$ and `0x7001` $+2n$. Set the layer field of every unused entry to 3.
4. Make sure the sprite layers in use are not disabled in the Base Configuration register.
5. Each frame, during vertical blanking, rewrite the `X` and `Y` fields of the sprites that moved.

9.9 Notes

- Scanning the 128 entries needs one read of the table per entry, at most 128 of the 512 PPU cycles of a line, from internal memory.
- Sprites are not clipped to a window, and there is no priority bit per sprite beyond the layer.

CHAPTER 10

Timing

10.1 Clocks

The device runs from the system's master clock. It does one PPU cycle every two master clocks, and one pixel every two PPU cycles, which is four master clocks. Table 10.1 gives the rates for the master clock of the SIRCUS system, 21,477,272 Hz, which is the NTSC colour subcarrier frequency multiplied by six.

Clock	Master clocks	At 21,477,272 Hz
Master clock	1	21,477,272 Hz
PPU cycle	2	10,738,636 Hz
Pixel	4	5,369,318 Hz
Line	1364	15,746 Hz
Frame	357,368	60.1 Hz

Table 10.1: Clock Rates

10.2 The CPU Clock

The CPU advances one execution phase on each rising edge of `CLKI` (*SIRC-1 Hardware Reference Manual*). The system supplies the CPU and the video device with the same `CLKI`, which is the only clock the device has, and `CLKI` is one PPU cycle: the master clock divided by two, 10,738,636 Hz at 21,477,272 Hz. The figures that follow are consequences:

- An ordinary instruction takes 6 `CLKI` cycles, 12 master clocks, so the CPU can run about 1.79 million instructions a second.
- A line is 682 `CLKI` cycles, about 113 instructions, and a frame is 178,684 cycles. NTSC vertical blanking, 38 lines, is 25,916 cycles, about 4,300 instructions.
- The registers are sampled 228 master clocks, 114 cycles, after the start of a line. A line-compare interrupt is recognised at an instruction boundary, so the handler's first instruction is fetched at best 6 and at worst about 12 cycles later (*SIRC-1 Architecture Reference Manual*, interrupt latency; more after a coprocessor call or a DMA transfer). That leaves about 100 cycles, some 16 instructions, for a handler to change a register before it takes effect on the same line.
- A `DMAT` of 255 words takes 522 cycles, three quarters of a line, and cannot be interrupted.

The video device never waits for the CPU, and it gives the CPU no wait states: every access to the device completes in the cycle it is made (Section 3.3.1).

10.3 Line Timing

A line is 1364 master clocks, 682 PPU cycles, in three parts. A line begins with the horizontal sync pulse (Section 11.5), and the picture follows the back porch; the front porch ends the line. The line timing is fixed and the same in both modes:

Part	Master clocks	PPU cycles	Purpose
Sync and back porch	260	130	Nothing is drawn; Mode 2 prefetches
Visible	1024	512	256 pixels at 4 master clocks each
Front porch	80	40	Nothing is drawn
Line	**1364**	**682**	

Table 10.2: Line Timing

The visible time of a line is a whole number of master clocks per pixel: four.

10.3.1 The First Line

Line 0 is always blank: the device shows black for the whole of it, whatever the registers hold, and fetches nothing for its background layers. The sprites of a line are fetched during the previous line, and line 0 has no previous line in which video memory is free, because vertical blanking belongs to the CPU. Line 0 therefore does the work of that previous line: it scans the sprite table and fetches the sprites for line 1 as any other line does for the next. The picture is 256 by 224 pixels in lines 0 to 223, of which line 0 is black, so 223 lines show a picture. Programs should keep nothing of importance on the first line of the picture; a television normally hides it in overscan.

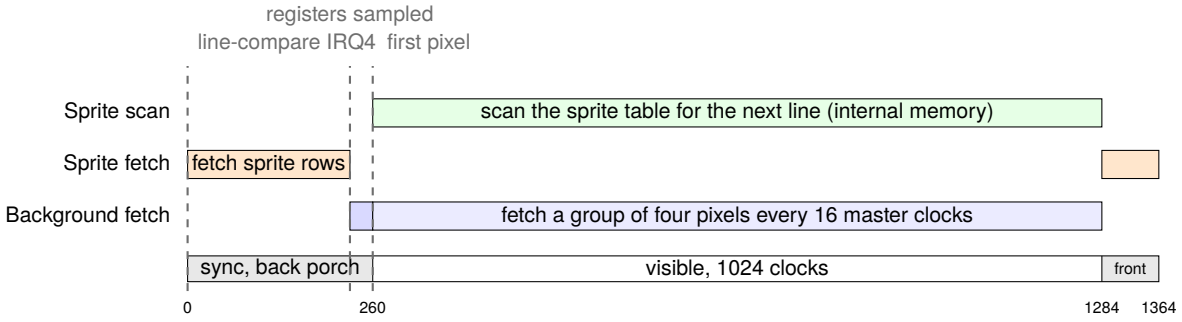


Figure 10.1: Timeline of One Visible Line, in Master Clocks

10.4 Frame Timing

A frame is 262 lines on NTSC and 312 on PAL. Lines 0 to 223 are visible and draw the picture. The remaining lines, 38 on NTSC and 88 on PAL, are the vertical blanking interval, in which nothing is drawn and the CPU may update video memory (Section 3.3.1). The device fetches nothing in these lines.

Table 10.3 gives the times. They follow from the clocks: on NTSC the PPU cycle is 93.1 ns and a pixel, four master clocks, is 186.2 ns.

	NTSC	PAL
Master clock	21.477 MHz	21.281 MHz
Line	63.5 μs	64.1 μs
Frame	16.64 ms	20.00 ms
Frame rate	60.10 Hz	50.00 Hz
Vertical blanking	2.41 ms	5.64 ms
Pixel	186.2 ns	188.0 ns
Visible part of a line	47.7 μs	48.1 μs

Table 10.3: Video Timing

10.4.1 PAL

The OUT bit of the Status register says which the device is. PAL has the same picture, 256 by 224 pixels in lines 0 to 223, at the top, and the same line timing of 1364 master clocks, but 312 lines per frame, so the vertical blanking interval is longer. A program written for NTSC runs unchanged on PAL, with more time in vertical blanking and a lower frame rate. With a PAL master clock of 21,281,370 Hz (the PAL colour subcarrier multiplied by 4.8) a frame is 425,568 master clocks, 50.0 Hz. The region is fixed when the device is made and is read from OUT.

10.5 Interrupts

The device has two interrupt sources, on different lines, so a handler knows which it is serving. The device races the beam: it never waits for the CPU, so an interrupt is raised at a fixed point of the frame whether or not the CPU is ready, and the time the CPU takes to respond is the program's concern. A program that shares $\overline{\text{IRQ4}}$ with another device can read the LIP bit of the Status register to see whether the device's interrupt is the one pending.

10.5.1 Vertical Sync

The vertical sync interrupt is the non-maskable interrupt, $\overline{\text{NMI}}$, Level 5. It is edge-triggered, so the device needs no acknowledgement: when VIE is set in the Interrupt Control register (Section 4.10), the device asserts the $\overline{\text{NMI}}$ pin for one PPU cycle at the first cycle of line 224, the start of vertical blanking. The program has the whole of vertical blanking to update video memory (Section 3.3.1). The pulse is exactly one CLKI cycle (Section 10.2). The CPU samples $\overline{\text{NMI}}$ on every rising edge of CLKI except during reset (*SIRC-1 Hardware Reference Manual*), so it does not miss the pulse. A pulse that arrives while the handler of the previous one is still running, or while a Level 5 request is pending, is not taken as an interrupt: it raises the Level Five Hardware Exception Conflict fault (*SIRC-1 Architecture Reference Manual*). The handler must therefore return before the next vertical sync, 16.6 ms later on NTSC and 20.0 ms on PAL.

The $\overline{\text{NMI}}$ cannot be masked, so a program must install its handler before it sets VIE. A DMA transfer cannot be interrupted, the $\overline{\text{NMI}}$ included. The edge is latched and the interrupt is taken when the transfer ends, so it is delayed, not lost, but a transfer that spans line 224 shortens the time the program has in vertical blanking by the delay. A program keeps its transfers short (Section 10.2 gives their lengths), or arranges for them to finish before line 224.

10.5.2 Line Compare

The line-compare interrupt is interrupt level 4, $\overline{\text{IRQ4}}$, the highest of the maskable levels. When LIE is set, the device asserts the line at the first cycle of the line whose number is in the LINE register (Section 4.9). It fires on any line of the frame, visible or not, so lines 0 to 223 can be used for raster effects. The $\overline{\text{IRQ4}}$ line is level-sensitive, so the device holds it asserted until the program

acknowledges it by writing any value to `LINE`, which also sets the line for the next interrupt, or by clearing `LIE`. The `LIP` bit of the Status register is set while the interrupt is pending. A `LINE` value at or beyond the number of lines in the frame never matches. If the line is reached again while an interrupt is still pending, nothing is queued: the line is already asserted.

The palette is writable at any time and a write changes the pixels drawn after it; the registers are sampled once per line, 32 master clocks before the first visible pixel in Mode 2 and Mode 3, so a register written within the 228 master clocks after the interrupt takes effect on that line, and later ones on the next. The device does not wait for the CPU, so how soon after the interrupt the CPU's write arrives is the program's concern, and a handler for a raster effect must be short and of known length.

10.6 Fetch Budget

Reading video memory is the device's limit: one read per PPU cycle, eight per group of four pixels. The modes spend the eight slots differently.

Use of the slots	Mode 1	Mode 2	Mode 3
Background tilemap entries	4	2–4	3–4
Background tile data words	4	2–4	3–4
Sprites	0	0	0
Total, typical (Mode 2, 3) or fixed (Mode 1)	**8**	**4.75 of 8**	**6.4 of 8**
Total, worst case	**8**	**8**	**8**

Table 10.4: Fetch Slots per Group of Four Pixels

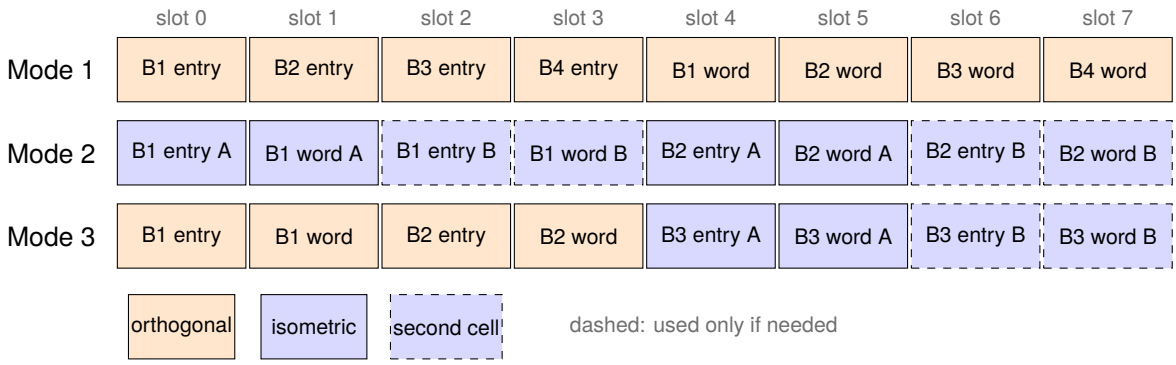


Figure 10.2: Who Uses Each Fetch Slot of a Group of Four Pixels

The status layer of Mode 2 takes its reads from the slots the isometric backgrounds leave spare, through a queue (Section 7.6); they are not in the table. Mode 3's orthogonal backgrounds read a tilemap entry and a tile word in every group, at fixed slots (Chapter 8); with 2-bit tiles they read only in every second group. A mode with more backgrounds or a different mix of fetches than these needs more reads per group, such a queue, or a faster memory.

10.7 Pixel Pipeline

The device draws a group of four pixels in two stages. During one group the device fetches the data for a later group and, at the start of the group, combines the data it already has into four finished pixels, which are shown one at a time over the group. Mode 1 combines the fetches of the previous

group; Mode 2 combines the data it prefetched (Section 7.5). The first pixels of a line in Mode 1 are at the position the scroll registers give (Section 6.2.1).

Figure 10.3 shows the schedule of one group in Modes 1 and 3, one read to a `CLKI` cycle, with the four pixels that are shown while it is made. A pixel lasts two cycles. In Mode 3 the reads of the second cell of the isometric background are made only when the group straddles two cells.

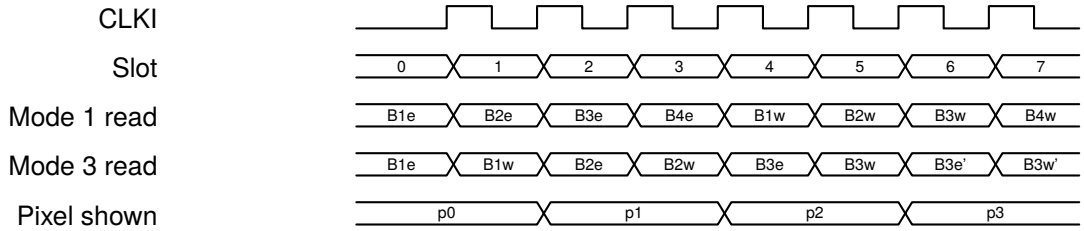


Figure 10.3: Fetch Schedule of One Group of Four Pixels (e: Tilemap Entry, w: Tile Word, ′: Second Cell)

CHAPTER 11

Signal Timing

This chapter gives the timing of the pins of the device (Chapter 2): the bus cycles in which the CPU reads and writes it, the interrupt outputs, reset, and the video output. Chapter 10 gives the timing inside the device, in the same units.

11.1 Reading the Figures

The figures follow the conventions of the bus timing diagrams of the *SIRC-1 Hardware Reference Manual*. Each labeled interval is one `CLKI` cycle, which is one PPU cycle, seen between rising edges. Waveform rows show physical levels: for an active-low signal, `L` means asserted. Bands show a value; an empty band carries no defined value, and a three-state signal that is not driven is drawn as a flat line at mid-level. The pulses in the `Sample` row mark the rising edges at which the device samples the bus. Electrical delays are not specified, so the figures show logical timing only.

11.2 Bus Cycles

The device answers every cycle in which $\overline{\text{CS}}$ and $\overline{\text{BAS}}$ are both asserted, and it answers in the cycle itself: $\overline{\text{BACK}}$ is asserted from the start of the cycle, so the CPU accepts the cycle at the next rising edge and never waits (Chapter 2).

11.2.1 Writes

The CPU presents the offset on A0–A15, the data on D0–D15 and BRW low, just after the rising edge that starts the cycle. The device takes the data at the rising edge that ends the cycle; the register, palette entry, video memory word or sprite table word holds the new value from that edge, and a read in the next cycle returns it. A write to an offset that is not stored, or one that the rules of Section 3.3.1 discard, is acknowledged in the same way.

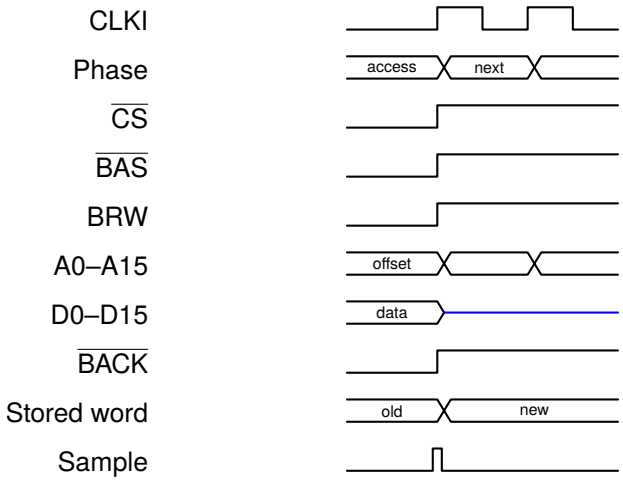


Figure 11.1: Write Cycle

11.2.2 Reads

On a read the device drives D0–D15 with the word the offset selects, from the start of the cycle to its end, and releases the bus after it. The word is valid before the rising edge that ends the cycle,

which is the edge at which the CPU samples it. A read changes nothing in the device; no register has a side effect on a read.

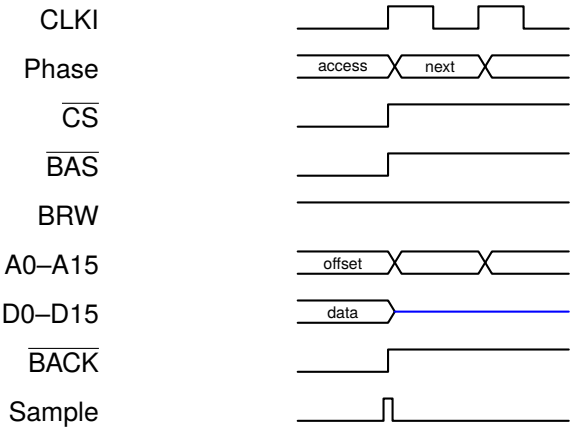


Figure 11.2: Read Cycle

11.2.3 Consecutive Cycles

The device accepts one access in every cycle, with no recovery time between accesses. A DMA transfer, which makes one bus cycle for each word (*SIRC-1 Hardware Reference Manual*), therefore accepts one word per `CLKI` cycle into the palette, the sprite table or video memory, whichever the offsets address, during the write part of the transfer. That is the rate of the writes only: a whole copy also makes source reads and setup cycles, so its rate is lower (*SIRC-1 Hardware Reference Manual*; Section 10.2 gives the time of a `DMAT`).

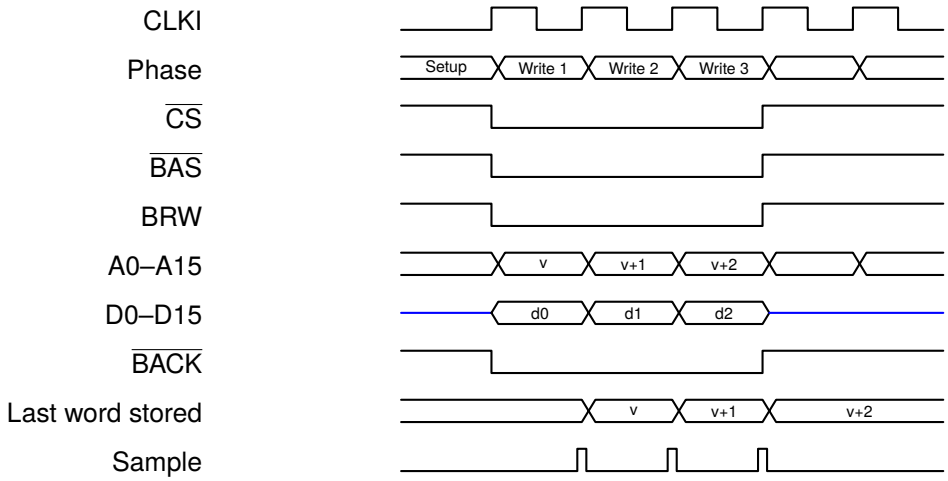


Figure 11.3: Three-Word DMA Write into the Device

11.2.4 The Start of Vertical Blanking

Video memory and the sprite table are open to the CPU from the first cycle of line 224 to the last cycle of line 261, and at any time while `GD` is set (Section 3.3.1). The gate changes at a line boundary, so it is exact to the cycle. A write in the last cycle of line 223 is discarded; a write in the first cycle of line 224 is stored. The vertical sync interrupt is raised in that same first cycle (Section 10.5), and the CPU takes it at the end of the instruction in progress, so the handler's first access to video memory is some cycles later than the figure shows.

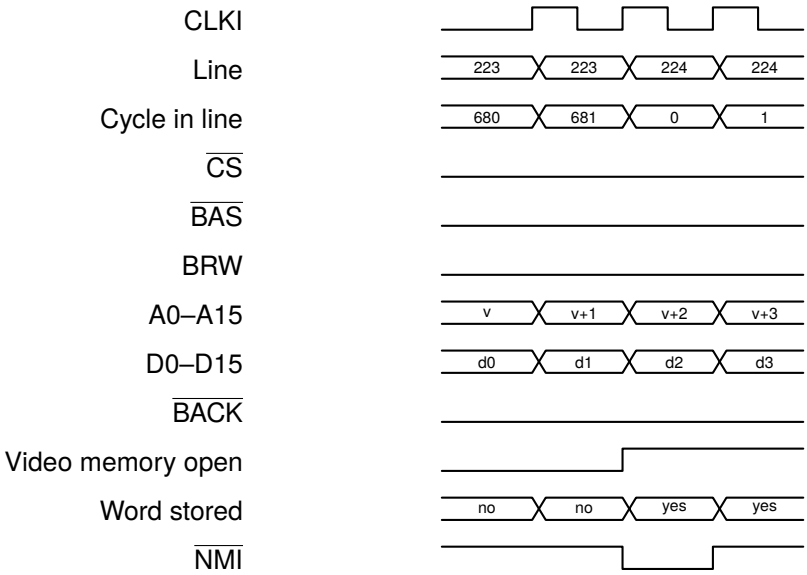


Figure 11.4: Video Memory Writes across the Start of Vertical Blanking (VIE set)

11.3 Interrupt Timing

The interrupt outputs change just after a rising edge of CLKI, like the other outputs of the device. The CPU samples them at the rising edge that ends the cycle.

11.3.1 Vertical Sync

Figure 11.4 shows the pulse: NMI is asserted for the whole of the first cycle of line 224 and deasserted in the next, when VIE is set in the Interrupt Control register. The pulse is one CLKI cycle, which the CPU samples on every rising edge except during reset, and the device does not wait for or expect an acknowledgement. The handler must return before the next pulse, or the CPU raises a conflict fault (Section 10.5).

11.3.2 Line Compare

IRQ4 is asserted at the first cycle of the line whose number is in LINE, when LIE is set, and is held. It is released by the program, with a write to LINE, or by clearing LIE. The LIP bit of the Status register follows the output.

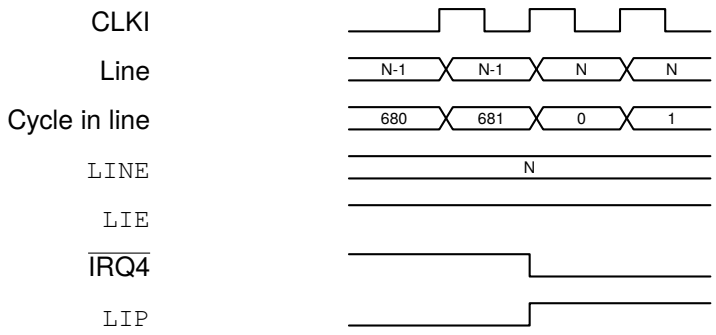


Figure 11.5: Line-Compare Interrupt Raised at the Start of Line N

The acknowledging write is an ordinary write cycle. The device releases IRQ4 and clears LIP at the rising edge that ends the write, so the line is high in the cycle after it. A program should write LINE early in its handler, and before it returns, because IRQ4 is level-sensitive and the CPU recognises the interrupt again if the line is still asserted when the handler ends (Section 10.5).

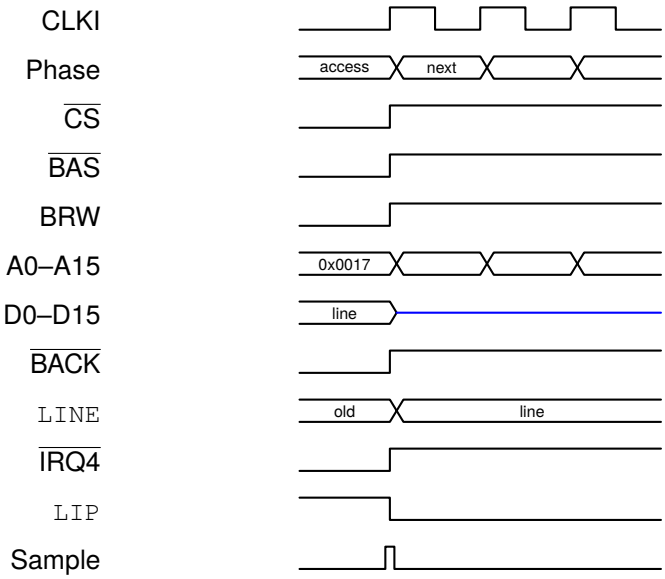


Figure 11.6: Acknowledging the Line-Compare Interrupt with a Write to LINE

11.4 Reset

RST is sampled at each rising edge of CLKI. Every register takes its reset value at the first edge at which RST is sampled asserted (Table 4.6), and the outputs go to their idle state: the interrupt outputs are deasserted, CSYNC is high, and the colour outputs are at black. The line and cycle counters are held at the first cycle of line 0 while RST is asserted. The first cycle of line 0 is the cycle that follows the rising edge at which RST is first sampled deasserted. The contents of the palette, video memory and sprite table are undefined after reset (Chapter 4).

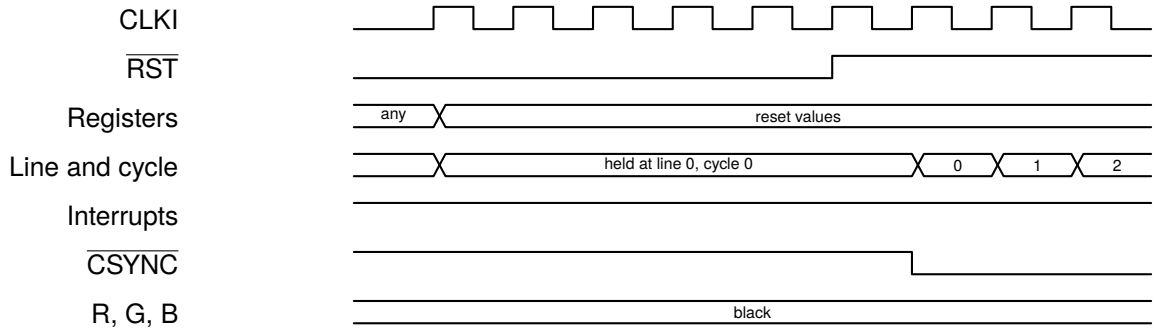


Figure 11.7: Reset, with RST Held for the Minimum of Six Cycles

11.5 Video Output

The video outputs follow the line timing of Chapter 10. A new pixel appears on R, G and B every four master clocks, at the pixel boundary, and the colour outputs are at black outside the visible part of the line, in the vertical blanking interval, and in forced blank. The brightness setting scales the colour before conversion (Section 4.2).

11.5.1 Horizontal Sync

CSYNC is asserted for the first 100 master clocks (50 cycles) of every line, and is high for the remaining 1264. The pulse begins at the first cycle of the line, the same cycle at which the line-compare interrupt is raised. The back porch follows it, and the first pixel is 160 master clocks after the end of the pulse. The line ends with the front porch, 80 master clocks; the 64 clocks before the

first pixel and the 44 before the front porch are black, so the picture has a narrow black border on each side.

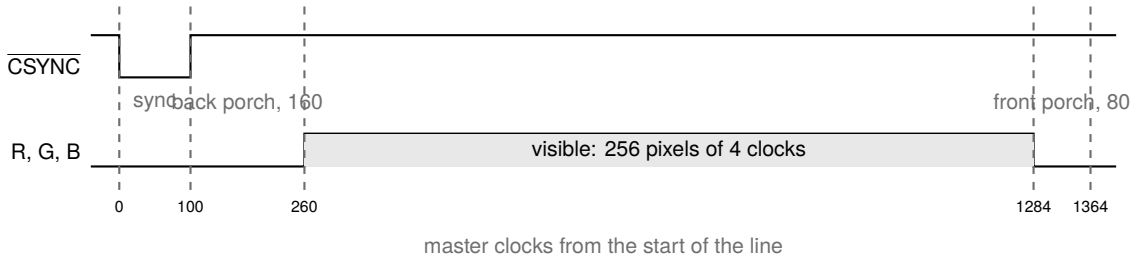


Figure 11.8: Horizontal Sync and the Visible Part of a Line

11.5.2 Vertical Sync

Lines 0 to 223 are the picture, and line 0 is always blank (Section 10.3.1). The vertical sync is three lines long and ends 14 lines before the first line of the picture: lines 245, 246 and 247 on NTSC, and lines 295, 296 and 297 on PAL. In each of those lines $\overline{\text{CSYNC}}$ is asserted from the first cycle of the line for 1272 master clocks (636 cycles) and is high for the last 92. The other lines of the vertical blanking interval carry the ordinary horizontal sync pulse, and there are no equalising pulses.



Figure 11.9: Vertical Sync in Lines 245 to 247 (NTSC), with Horizontal Sync before and after

CHAPTER 12

Programming the Device

This chapter collects what a program does in what order. It adds no new rules; each rule is stated where the feature is described, and the sections of the earlier chapters are cited.

12.1 Starting Up

At reset the picture is blank (`GD` is 1), every writable register is zero except `CONFIG` (Section 4.11), and video memory, the palette and the sprite table are undefined. A program starts the device in this order:

1. Install the handler for the vertical sync interrupt, $\overline{\text{NMI}}$, before anything enables it (Section 10.5).
2. With the picture still blank, write the palette (`0x6000`–`0x60FF`), the tile data and the tilemaps into video memory (Chapter 3). Video memory is open to the CPU while `GD` is 1.
3. Fill the sprite table, hiding every entry the program does not use by setting its layer field to 3 (Section 9.2).
4. Write the tilemap address, tile data address, palette configuration and scroll registers of the backgrounds in use, the `STILES` register, and the `TILESIZE` register.
5. Write `INTCTL` to enable the interrupts the program uses (`VIE`, and `LIE` with `LINE` for a line-compare interrupt).
6. Write `CONFIG` with the mode, the brightness and `GD` = 0. The picture appears from the next line.

12.2 One Frame

Figure 12.1 shows a frame as the program sees it. The device draws lines 0 to 223 without help; line 0 is always blank (Section 10.3.1). At the first cycle of line 224 it raises the $\overline{\text{NMI}}$ (if `VIE` is set) and sets the `VBL` bit of the Status register, and the handler has the rest of vertical blanking, 38 lines or 25,916 cycles on NTSC and 88 lines on PAL, to change video memory, the sprite table and the registers for the next frame.

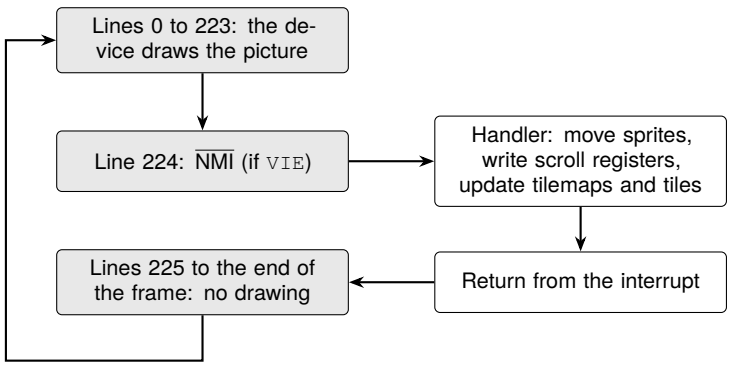


Figure 12.1: A Frame

A program that does not use the interrupt waits for the VBL bit instead: it reads STATUS until bit 7 is set, and it knows how far into the frame it is from VLINE. A program that polls must finish before VBL falls at line 0, so it reads VBL again before each write that matters near the end.

A DMAT of 255 words takes 522 cycles (Section 10.2), so a program doing nothing else could move at most about 12,600 words in an NTSC vertical blanking interval. A program that needs more than it can move in one frame spreads the work over several frames, or turns the picture off (GD = 1) for the time it needs.

12.3 Programming Cautions

1. **Video memory.** A write to video memory or to the sprite table outside vertical blanking is lost, and a read returns zero, unless GD is 1 (Section 3.3.1). A program knows video memory is open when VBL in the Status register is set. The palette and the registers can be written at any time.

2. **The sprite table.** The table is undefined at reset. A program must hide the entries it does not use before it clears GD, or random sprites appear.

3. **NMI.** The $\overline{\text{NMI}}$ cannot be masked. Install the handler before setting VIE. A DMAT that is running at line 224 delays the interrupt until it ends (Section 10.5).

4. **Line compare.** The $\overline{\text{IRQ4}}$ line stays asserted until the program writes LINE. A handler that does not write LINE, or clear LIE, is entered again as soon as it returns.

5. **Registers during the picture.** The device samples the scroll registers once per line, and a change made during a line takes effect on the next line. It reads the tilemap address, tile data address and palette configuration registers as it fetches, so a change during the visible part of a line can mix old and new settings within that line: change them only in vertical blanking or while GD is 1 (Chapter 7).

6. **Changing mode.** Change MODE only in vertical blanking or while GD is 1. The values 3 to 7 of MODE are reserved and must not be written.

7. **Reserved registers.** Software must not write the reserved registers and reserved bits (Chapter 4). A read of an open bus offset returns zero.

8. **Scroll registers.** The scroll registers are 10 bits. A program keeps its position in a wider variable and writes the low 10 bits; in Mode 2 it also adjusts SCRY when it brings SCRX back past 1024 (Section 7.3.1).

9. **Sprites per line.** A line shows at most 32 sprites; the rest are dropped and SOV is set. Read SOV in vertical blanking (Section 9.6).

10. **Transparent pixels.** Pixel value 0 of a background or a sprite is transparent. If every layer is transparent at a pixel, the device shows palette entry 0 (Section 5.7).

11. **PAL and NTSC.** A program that depends on the number of lines or the frame rate reads OUT in the Status register.

CHAPTER A

Quick Reference

A.1 Registers

Offset	Name	Fields
0x0000	CONFIG	15 GD; 14–11 B1D–B4D; 10–8 S1D–S3D; 7 SHD; 6–3 BRT; 2–0 MODE (0, 1, 2 for Modes 1, 2, 3)
0x0001	TILESIZE	15–12 B1S–B4S; 11–9 S1S–S3S; 7–4 B1X–B4X; 3–0 B1Y–B4Y
0x0002–0x0005	B1MAP–B4MAP	Tilemap word address
0x0006–0x0009	B1TILES–B4TILES	Tile data word address
0x000A–0x000D	B1SCRX–B4SCRX	9–0 scroll amount
0x000E–0x0011	B1SCRY–B4SCRY	9–0 scroll amount
0x0012–0x0015	B1PAL–B4PAL	15–8 OFFSET; 2 T2; 1–0 SIZE
0x0016	INTCTL	1 LIE; 0 VIE
0x0017	LINE	8–0 line for the line-compare interrupt; a write acknowledges it
0x0018	STILES	Sprite tile data word address
0x0019	STATUS	7 VBL; 6 LIP; 5 SOV; 4 OUT; 3–0 VER (read-only)
0x001A	VLINE	8–0 current line (read-only)

Table A.1: Register Summary

A.2 Memory Map

Registers 0x0000–0x00FF; palette 0x6000–0x60FF; sprite table 0x7000–0x70FF; video memory 0x8000–0xFFFF (32,768 words). Everything else is open bus.

A.3 Formats

Colour: bit 15 reserved; bits 14–10 red; bits 9–5 green; bits 4–0 blue.

Tilemap entry: bit 15 V; bit 14 H; bit 13 P; bits 12–10 palette selection; bits 9–0 tile number.

Tile data word: four 4-bit pixels, leftmost in bits 15–12. Value 0 is transparent.

Isometric tilemap entry flips: V darkens, H lightens (both: normal).

2-bit orthogonal tile: 8 words, one per row, leftmost pixel in bits 15–14; word at base $+ 8t + r$.

Sprite shadow: value 15 of sprite palette 7 darkens what is behind its layer.

Palette entry: $((s \ll k) + \mathrm{OFFSET} + v) \bmod 256$, with k = 2, 3, 4, 5 for SIZE 0, 1, 2, 3.

A.4 Orthogonal Tiles

16 by 16 tile, 4 bits: 64 words, quadrants top left, top right, bottom left, bottom right; word at base $+ 64t + 16q + 2(r \bmod 8) + h$.

Tilemap sizes: screens of 32 by 32 entries, A B over C D, 1,024 words apart.

Priority bit: 1 puts the tile in front of all sprites.

Reset: `CONFIG` = `0x8078` (blank, full brightness, Mode 1); the other writable registers are 0, and `STATUS` reads `0x0001` on NTSC and `0x0011` on PAL.

A.5 Mode 1 Addresses

Tilemap entry: tilemap base $+ \lfloor x/8 \rfloor + 32\lfloor y/8 \rfloor$.

Tile data word: tile data base $+ 16t + 2r + h$ for tile t, row r and half h of the row.

A.6 Mode 2 Formulas

For the plane pixel $(X, Y) = (x + \texttt{SCRX},\ y + \texttt{SCRY})$:

$$u = Y + \lfloor X/2 \rfloor, \qquad v = Y - \lfloor (X+1)/2 \rfloor,$$
$$c_u = \lfloor u/16 \rfloor \bmod W, \qquad c_v = \lfloor v/16 \rfloor \bmod H,$$

with W and H the map's width and height in cells (32 or 64), and the tilemap entry at tilemap base $+ 1024s + 32(c_v \bmod 32) + (c_u \bmod 32)$, where $s = \lfloor c_v/32 \rfloor (W/32) + \lfloor c_u/32 \rfloor$ is the screen.

For the texel $(t_u, t_v) = (u \bmod 16, v \bmod 16)$ of a tile:

$$i = (t_u - t_v) \bmod 4,$$
$$(a, b) = (t_u - \lfloor i/2 \rfloor,\ t_v + [0, 1, 1, 2]_i),$$
$$w = 16\lfloor ((a - b) \bmod 16)/4 \rfloor + (a \bmod 16),$$

the word w of the tile, at tile data base $+ 64t + w$, nibble i (0 is bits 15–12).

The top pixel of cell (c_u, c_v) is at the plane pixel $(16(c_u - c_v),\ 8(c_u + c_v))$.

SIRCADA Reference Manual

This manual describes SIRCADA, the SIRC Analogue Display Adapter: the picture processing unit that turns tiles, tilemaps and palette entries in its video memory into a 256 by 224 pixel picture. It is written for the programmers who draw with it, the authors of the tools that prepare graphics for it, and the engineers who implement it. What the CPU does when it reads and writes the device is the subject of the companion manuals.

This manual contains:

- The device's memory map, registers and data formats
- Mode 1, three orthogonal tiled backgrounds, and Mode 2, two isometric backgrounds
- The scanline timing and the video memory fetch schedule of each mode
- The implementation status of every feature and the questions still open

SILICON INTEGRATED RESEARCH CORPORATION